

Projet de fin d'études présenté par

Youenn LE JEUNE

Élève-ingénieur de l'INSA Rennes

Spécialité INFO

2025-2026

MAINTIEN EN CONDITION DE
SÉCURITÉ DE LF ENERGY SEAPATH

Lieu du Projet de Fin d'Études

Savoir-faire Linux

Tuteur du Projet de Fin d'Études

Erwann ROUSSY

Correspondante pédagogique INSA

Barbara FILA

PFE soutenu le 2026-06-17

AUTORISATION DE DIFFUSION ET D'ARCHIVAGE DU RAPPORT

NOM et Prénom de l'auteur : Youenn LE JEUNE

Spécialité : INFO

Titre du projet : Maintien en condition de sécurité de LF Energy SEAPATH

Nom de l'entreprise : Savoir-faire Linux

NOM et Prénom du tuteur du Projet de Fin d'Études : Erwann ROUSSY

NOM et Prénom du correspondant pédagogique INSA : Barbara FILA

Archivage du rapport de PFE

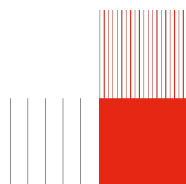
A l'issue de son stage, l'étudiant(e) stagiaire rédigera un rapport qui devra être communiqué aussi bien à l'organisme d'accueil qu'à l'établissement d'enseignement supérieur pour évaluation.

Par obligation réglementaire (instruction 2005-003 parue au B.O du 16 juin 2005), l'INSA Rennes est tenu de conserver pour archive une version du rapport de PFE. Tout rapport de PFE sera donc systématiquement archivé par le département de l'auteur de l'INSA de Rennes.

Si le rapport contient des données confidentielles, une version expurgée pourra être archivée à la place de la version intégrale.

☒ archivage de la version intégrale

☐ archivage de la version expurgée, merci de justifier :



Diffusion du rapport de PFE

Autorisation de diffusion par l'entreprise commanditaire

Nous, soussignés, représentant de l'entreprise commanditaire

Nom : Bail

Prénom : Eloi

Fonction : Directeur des opérations

☒ autorisons le signalement et la diffusion du document désigné ci-dessus sur l'intranet du département de l'auteur de l'INSA

☒ autorisons uniquement le signalement du document désigné ci-dessus sur l'intranet du département de l'auteur de l'INSA

☐ n'autorisons ni le signalement ni la diffusion du document désigné ci-dessus sur l'intranet du département de l'auteur de l'INSA et demandons la confidentialité (5 ans maximum), jusqu'à la date suivante (mois/année) :

Fait à Rennes , le 10/06/2026

Signature et/ou cachet de l'entreprise commanditaire :



Autorisation de diffusion par l'auteur

Je soussigné(e),

Nom :

Prénom :

☒ autorise le signalement et la diffusion du document désigné ci-dessus sur l'intranet du département de l'auteur de l'INSA

☐ autorise uniquement le signalement du document désigné ci-dessus sur l'intranet du département de l'auteur de l'INSA

☐ n'autorise ni le signalement ni la diffusion du document désigné ci-dessus sur l'intranet du département de l'auteur de l'INSA

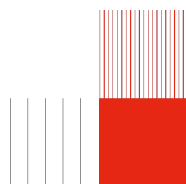
Fait à Rennes , le 10/06/2026

Signature :



Autorisation de diffusion par le correspondant pédagogique de l'INSA

Par défaut, le correspondant pédagogique, membre de l'équipe enseignante, est supposé avoir une contribution mineure sur le rapport et ne pas s'opposer à sa diffusion. Merci de contacter le responsable des stages du département en cas d'opposition.



REMERCIEMENTS

Tout d'abord, merci beaucoup à mon tuteur de stage, Erwann, de m'avoir fait confiance pour ce stage. Merci pour ton aide, merci aussi de ne pas trop m'avoir aidé et de m'avoir laissé galérer, c'était très formateur. Merci aussi pour les discussions sans trop tête ni queue pour laisser notre cerveau s'arrêter un peu parfois. D'ailleurs, il manque les ravioli dans ta classification.

Merci à la direction de faire le maximum pour que les stagiaires soient intégrés dans la vie professionnelle de l'entreprise, en nous faisant participer aux réunions quotidiennes et en nous poussant à contribuer aux projets internes.

Merci aux collègues de Montréal de l'équipe VulnScout qui m'ont intégré à leurs cycles de développement, ont corrigé mes bugs et revu mes PRs sans rechigner. Merci beaucoup à Valentin d'avoir tout fait pour que je sois bien intégré dans l'équipe, même à 5 000 km d'ici !

Enfin, merci à tous les collègues de Rennes pour leur bon accueil et notre intégration aisée à Florent et à moi, c'était sans prise de tête et ce fut un plaisir. À l'heure où j'écris ces lignes je n'en suis pas encore certain, mais je suis presque sûr qu'on va gagner le tournoi de Mölkky. Allez Kevin, allez Youssef on va y arriver !

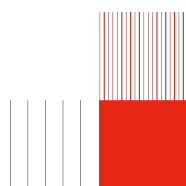
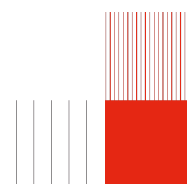


TABLE DES MATIÈRES

1	Introduction	1
2	Cadre et contexte du stage	1
2.1	Savoir-faire Linux	1
2.1.1	Présentation de Savoir-faire Linux	1
2.1.2	Outils et méthodes de travail	2
2.2	Le projet SEAPATH	2
2.2.1	Le réseau électrique	2
2.2.2	Les limites du modèle actuel	3
2.2.3	La virtualisation	4
2.2.4	Architecture de SEAPATH	4
2.2.5	Génération et déploiement	5
2.2.6	Aspects de cybersécurité	5
3	Vulnérabilités sur SEAPATH Yocto	6
3.1	État de l'art	6
3.1.1	Solutions avec SBOM	6
3.1.2	Solutions sans SBOM	6
3.2	Comparaison des solutions	7
3.2.1	Méthodologie	7
3.2.2	Nombre de vulnérabilités détectées	8
3.2.3	Nombre de vulnérabilités à évaluer	9
3.2.4	Choix final et compte-rendu	10
3.3	Mise en place de l'outil	10
3.4	Analyse des vulnérabilités détectées	11
3.5	Intégration Continue	13
3.5.1	Transfert sur GitHub Actions	13
3.5.2	Détection de vulnérabilités	14
3.5.3	Pull Requests	15
3.5.4	Approche multi-dépôts	17
3.5.5	Synchronisation entre dépôts	18
4	Vulnérabilités sur SEAPATH Debian	21
4.1	Approches de détection	21
4.2	Intégration du générateur de SBOM	21
4.3	Détection des vulnérabilités	23
4.4	Intégration en CI	24
4.5	Images de conteneurs	24
5	VulnScout	26
5.1	Analyse des vulnérabilités de SEAPATH	26
5.2	Intégration Continue dans SEAPATH	29
5.3	Amélioration de la qualité du code	30
5.4	Comparaison avec Dependency-Track	31
6	Conclusion	32
	Bibliographie	33
	Annexes	36



GLOSSAIRE

API – Application Programming Interface: Interface logicielle qui permet de «connecter» un logiciel ou un service à un autre logiciel ou service afin d'échanger des données et des fonctionnalités. 18, 19

CI – Intégration Continue: Ensemble de pratiques utilisées en génie logiciel consistant à vérifier à chaque modification de code source que le résultat des modifications ne produit pas de régression dans l'application développée. 1, 2, 10, 13, 14, 16, 17, 19, 20, 24, 25, 29, 30, 32

conteneur: Enveloppe virtuelle qui contient une application et tous les éléments dont elle a besoin pour fonctionner : bibliothèques, outils, fichiers... Peut être grossièrement considéré comme une [Machine Virtuelle](#) légère. 2, 21, 24

CPE – Common Platform Enumeration: Schéma de nommage structuré permettant d'identifier des systèmes informatiques, des logiciels, des paquets ou encore du matériel informatique. Peut contenir beaucoup de détails tels que le distributeur, la langue, etc. (à l'inverse du [Package-URL \(PURL\)](#)) 3, 6, 7, 8

CVSS – Common Vulnerability Scoring System: Un score indiquant à quel point une vulnérabilité est grave. Va de 0 à 10, 10 indiquant une vulnérabilité absolument critique. 11, 14

distribution: Ensemble de logiciels qui, associés à un noyau Linux, fournissent un système d'exploitation opérationnel. Ils ne sont pas nécessaires pour que le système démarre, mais offrent les services requis pour le cas d'usage souhaité. 1, 5, 6

EPSS – Exploit Prediction Scoring System: Un score indiquant la probabilité qu'une vulnérabilité soit exploitée. Va de 0 à 1 (ou de 0% à 100%). 11, 14, 29

hyperviseur: Logiciel ou matériel permettant de créer et d'exécuter des VMs. Il agit comme un gestionnaire de ressources et de supervision entre les systèmes d'exploitation invités et le matériel physique de l'hôte. 4, 4, 5

IED – Intelligent Electronic Device: Dispositif électronique utilisé dans les systèmes de gestion et de contrôle du réseau électrique. Ils sont capables de collecter des données en temps réel, de prendre des décisions autonomes et d'exécuter des actions telles que l'ouverture ou la fermeture de disjoncteurs en réponse à des conditions anormales du réseau. 1, 3

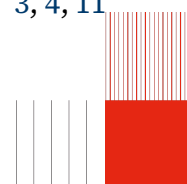
noyau: Cœur d'un système d'exploitation. Contient les programmes et pilotes essentiels à son fonctionnement, comme la gestion de la mémoire, des processus ou des I/O basiques. Souvent appelé par le terme anglais *kernel*. 4, 6, 7, 8, 9, 10, 11

open source: Caractéristique d'un projet logiciel dont la licence respecte des critères précisément établis par l'Open Source Initiative, c'est-à-dire les possibilités de libre redistribution, d'accès au code source et de création de travaux dérivés. 1, 2, 3, 12, 13, 21, 23, 32

PR – Pull Request: Demande d'intégration de commits dans une autre base de code. Le développeur pousse sa branche et demande à ce qu'elle soit fusionnée dans une autre, souvent la branche principale du dépôt. 2, 13, 15, 16, 17, 18, 19, 20, 24, 26, 31

PURL – Package-URL: Schéma de nommage basé sur la syntaxe URL permettant d'identifier de manière unique un paquet logiciel, indépendamment de son écosystème ou médium de distribution (contrairement au [Common Platform Enumeration \(CPE\)](#)). 3, 7

RT – Temps Réel: Caractéristique d'un système pour lequel le respect des contraintes temporelles dans l'exécution des traitements est aussi important que le résultat de ces traitements. 3, 4, 11

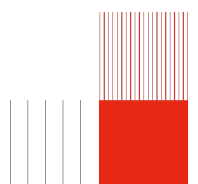


SBOM – Software Bill of Materials: Liste des composants utilisés pour créer un artéfact logiciel. Parfois appelée *nomenclature logicielle*. 2, 1, 6, 7, 8, 10, 14, 15, 16, 21, 22, 23, 24, 25, 26, 28, 29, 31, 32

smart grid: Réseau électrique modernisé qui utilise la technologie numérique, la communication bidirectionnelle et des commandes avancées pour optimiser la génération, la transmission et la distribution d'électricité. Il intègre notamment des sources d'énergie renouvelable pour améliorer l'efficacité et la fiabilité de la distribution. 3

TSC – Comité de Pilotage Technique: Comité gérant les aspects techniques d'un projet. Il se réunit régulièrement et ses réunions peuvent être ouvertes au public. Sigle issu de l'anglais *Technical Steering Committee*. 11, 24

VM – Machine Virtuelle: Émulation logicielle d'un ordinateur physique qui exécute un système d'exploitation et des applications comme un ordinateur physique, lancée sur un [hyperviseur](#). 3, 4, 5, 21



1 INTRODUCTION

Aujourd'hui, les infrastructures énergétiques évoluent rapidement, largement influencées par l'adoption croissante des énergies renouvelables, qui représentent désormais 15,8 % de la consommation d'énergie primaire en 2025, par rapport à seulement 9,6 % onze ans auparavant [1]. Cette transition entraîne un changement majeur dans la gestion des réseaux électriques, passant d'un modèle traditionnellement statique et unidirectionnel à un paradigme plus dynamique et imprévisible.

Au cœur de cette mutation se trouve la nécessité de repenser les systèmes de contrôle et d'automatisation, notamment en migrant des dispositifs physiques appelés **Intelligent Electronic Device (IED)** vers des versions virtuelles. Cette transition s'avère indispensable pour surmonter les défis associés aux coûts élevés d'exploitation et de déploiement des équipements physiques, ainsi qu'aux cycles de développement et de mise à jour complexes et chronophages.

Pour répondre à ces enjeux, une approche axée sur la virtualisation des fonctionnalités, indépendamment des contraintes matérielles, s'impose. Dans cette perspective, le projet SEAPATH [2], lancé par la Linux Foundation Energy, émerge comme une initiative majeure visant à concevoir une plateforme open-source et temps réel permettant d'exécuter la virtualisation des **IEDs** provenant de divers fournisseurs du secteur énergétique.

Les réseaux électriques faisant partie des infrastructures critiques d'un pays, il y a un très grand enjeu de sécurité : les acteurs du secteur de l'énergie doivent pouvoir suivre toutes les vulnérabilités qui affectent les logiciels utilisés pour gérer leur réseau.

Ce projet de fin d'études a principalement porté sur le monitoring des vulnérabilités logicielles de deux versions de SEAPATH, en se concentrant spécifiquement sur la génération de **Software Bills of Materials (SBOMs)**, l'analyse des vulnérabilités critiques et l'intégration en **Intégration Continue (CI)**. Il a mené à de nombreuses contributions dans plusieurs projets **open source** dont VulnScout [3], un outil spécifiquement créé pour évaluer les vulnérabilités.

Après avoir posé le contexte du stage au Chapitre 2, nous aborderons l'étude des vulnérabilités sur la version Yocto de SEAPATH au Chapitre 3, avec une vue d'ensemble détaillée des travaux d'intégration en **CI** dans le Chapitre 3.5. Nous étudierons ensuite la mise en place de détection de vulnérabilités sur la version Debian [4] de SEAPATH au Chapitre 4. Enfin, le Chapitre 5 détaillera l'utilisation de VulnScout ainsi que les contributions apportées au projet.

2 CADRE ET CONTEXTE DU STAGE

2.1 Savoir-faire Linux

2.1.1 Présentation de Savoir-faire Linux

Savoir-faire Linux est une entreprise d'origine québécoise dont la principale activité est l'intégration de systèmes Linux [5] ou d'applications sous Linux. Fondée en 1999 à Montréal, où l'entreprise possède son siège social, elle s'est depuis étendue en France. Depuis 2018, c'est à Rennes que la branche française est installée. Proche du parc Oberthür, c'est dans ces bureaux que s'est déroulé ce stage.

Le bureau rennais est orienté Linux embarqué. Les projets portent sur l'intégration de **distributions** Linux ou d'applications sur des produits clients, de l'audit ou encore du développement.

L'entreprise participe aussi activement à la communauté open source, via des contributions à des projets et la participation à de nombreux salons. Enfin, plusieurs logiciels libres, comme Jami [6], SEAPATH [2] ou encore VulnScout [3], sont développés et maintenus par Savoir-faire Linux.

2.1.2 Outils et méthodes de travail

Chaque projet chez Savoir-faire Linux s'appuie sur un ensemble d'outils communs pour maintenir une organisation et une qualité de travail optimales. L'objectif est de détecter rapidement les erreurs et d'automatiser les tâches répétitives.

Pour collaborer efficacement, chaque projet est versionné dans des dépôts *Git* [7]. Chez Savoir-faire Linux, l'application *Gerrit* [8] héberge les dépôts, et met en place un système de revue de code. Chaque commit est vérifié par une (ou plusieurs, selon les projets) autres personnes de l'entreprise, ce qui permet de détecter les erreurs ou de proposer de meilleures approches. Lorsque la personne faisant la revue demande ou propose des modifications, le commit en question doit être modifié, afin de garder un historique parfaitement cohérent. Un commit n'est ajouté dans la branche principale d'un dépôt que lorsque la revue est validée. Certains projets [open source](#) comme SEAPATH ou VulnScout utilisent plutôt *GitHub* [9], où le même principe de revue de code est appliqué.

L'intégration continue, ou [CI](#), ajoute un deuxième filet de sécurité. C'est un ensemble de services automatisés pour compiler et tester du code. L'entreprise utilise l'application Jenkins [10] pour les projets utilisant *Gerrit*, ou bien le système *GitHub Actions* [11] pour les autres. À chaque nouveau commit ou [Pull Request \(PR\)](#), un cycle de [CI](#) commence et va effectuer diverses tâches : compiler le projet, lancer des tests, etc. Cela augmente les chances de détecter une régression, de rapidement régler les bugs, et d'éviter une longue investigation de problème plus tard. Sur des projets applicatifs, des tests d'analyse statique du code permettent aussi de maintenir une bonne qualité de code.

En parallèle, Savoir-faire Linux développe et maintient plusieurs outils facilitant le développement des projets. Ces outils sont utilisés en interne mais sont également disponibles en [open source](#). Parmi les plus importants, on peut noter :

- *CQFD* [12], permet de lancer facilement des commandes dans un [conteneur](#) ayant accès au dossier courant. Il est notamment utilisé pour configurer et utiliser le projet Yocto avec ses nombreuses dépendances, peu importe l'environnement d'exécution.
- *Cukinia* [13], un outil de test pour vérifier la configuration d'un système Linux au runtime : présence d'utilisateurs, de partitions de système de fichier, de programmes, de périphériques... Il permet d'appliquer une méthodologie de développement piloté par les tests pour l'intégration d'un système Linux complet.

Concernant l'organisation interne, les informations importantes sont diffusées par e-mails. Pour le quotidien, une instance *Mattermost* est utilisée comme plateforme de communication instantanée. La majorité du travail est faite en présentiel, mais il arrive de faire des réunions par visioconférence, notamment avec l'équipe de Montréal lorsque les deux bureaux travaillent sur des projets communs.

2.2 Le projet SEAPATH

2.2.1 Le réseau électrique

Le réseau électrique constitue l'ensemble des infrastructures permettant d'acheminer l'énergie électrique depuis les lieux de production vers les consommateurs. Il est constitué de divers

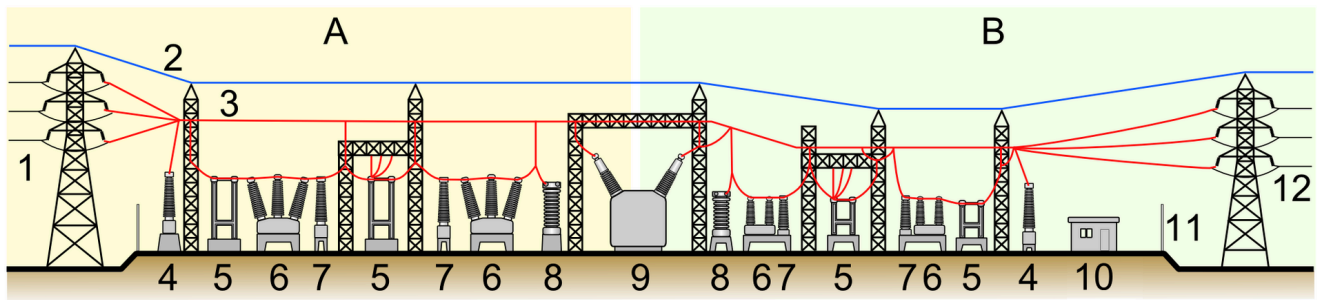


Figure 1 : Vue en coupe d'une sous-station électrique [14]

points de connexion appelés **nœuds**, qui correspondent aux **postes électriques**. Ces postes se divisent généralement en deux catégories : les postes d'interconnexion, qui aiguillent les lignes de même tension, permettant par exemple d'isoler une ligne défectueuse ou à des fins de réorganisation du réseau. On a aussi les **postes de transformation**, qui ajustent la tension en l'élevant (par exemple, à la sortie des centrales) ou en l'abaissant (pour la distribution vers les consommateurs).

Dans un poste de transformation on trouve de nombreux équipements (Figure 1), certains destinés à la transformation de la tension tels que le **transformateur de puissance** (9) tandis que d'autres sont là pour protéger et isoler les composants, comme les **sectionneurs** (5) ou les **disjoncteurs** (6).

Le **bâtiment de contrôle** (10) abrite des équipements assurant la gestion de ces composants. Les systèmes de contrôle et d'automatisation des réseaux électriques jouent un rôle crucial dans cette gestion, en intégrant des technologies avancées pour superviser et optimiser la distribution de l'énergie. Parmi ces technologies, les **IEDs** occupent une place centrale.

Un **IED** est un élément clé d'un poste électrique, remplaçant les relais traditionnels et autres appareils par des dispositifs dotés de microprocesseurs et de communications avancées. Il protège les lignes, génère des événements, gère les régulateurs de tension et collecte des données critiques, permettant une prise de décision rapide pour la protection et la restauration du réseau. [15]

2.2.2 Les limites du modèle actuel

Aujourd'hui, en raison principalement de l'essor des énergies renouvelables, intégrant ainsi des sources d'énergie aux moyennes et basses tensions, on passe d'un modèle statique et unidirectionnel du réseau vers un modèle beaucoup plus dynamique et imprévisible, qu'on appelle **smart grid**.

Le cycle de développement des systèmes de contrôle et d'automatisation des réseaux électriques est donc en pleine transformation. Il est nécessaire de porter le code développé pour les **IEDs** physiques vers des **IEDs** virtuels. Les dispositifs physiques ont des coûts opérationnels et de déploiement élevés, avec des cycles de développement, de test et de mise à jour complexes et longs. Pour relever ces défis, un tournant dans la conception des systèmes s'impose. La virtualisation des fonctionnalités, en se détachant des implémentations dépendantes du matériel, facilite la transition d'un réseau électrique traditionnel vers une **smart grid** orientée logiciel [16].

C'est dans ce tournant que s'inscrit le projet SEAPATH (Software Enabled Automation Platform and Artifacts), une initiative de la Linux Foundation Energy qui vise à développer une plateforme **open source** et **Temps Réel** qui permettra d'exécuter la **virtualisation d'IEDs** provenant de divers

fournisseurs du secteur de l'énergie avec de fortes contraintes de **fiabilité** et de **disponibilité**. Les co-mainteneurs du projet sont Savoir-faire Linux et RTE international.

2.2.3 La virtualisation

La **virtualisation** regroupe les techniques matérielles et/ou logicielles qui permettent à une seule machine d'exécuter plusieurs systèmes d'exploitation et/ou applications de manière isolée, comme s'ils tournaient sur des machines distinctes. Pour ce faire, on utilise un **hyperviseur**. Celui-ci agit comme une couche intermédiaire entre le matériel de la machine hôte et les systèmes d'exploitation invités, souvent appelés **Machines Virtuelles (VMs)**.

La virtualisation offre de nombreux avantages :

- Efficacité des ressources : la virtualisation permet d'exécuter plusieurs applications sur une seule machine, optimisant l'utilisation du matériel.
- Déploiement rapide : la mise en place des **VMs** est plus rapide que l'installation de matériel, surtout avec l'automatisation.

SEAPATH utilise le module noyau KVM [17], permettant de convertir l'OS en un **hyperviseur** dit « de type 1 », ce qui signifie que les **VMs** s'exécutent directement sur le matériel, sans traverser le système d'exploitation hôte. De plus, SEAPATH utilise QEMU [18] en conjonction avec KVM. Dans cette configuration, KVM se charge des interactions avec le matériel tandis que QEMU émule les périphériques et gère l'environnement de la **VM**. Enfin, libvirt [19] est utilisé pour gérer et orchestrer les **VMs**.

2.2.4 Architecture de SEAPATH

L'architecture générale de SEAPATH est présentée en Figure 2 :

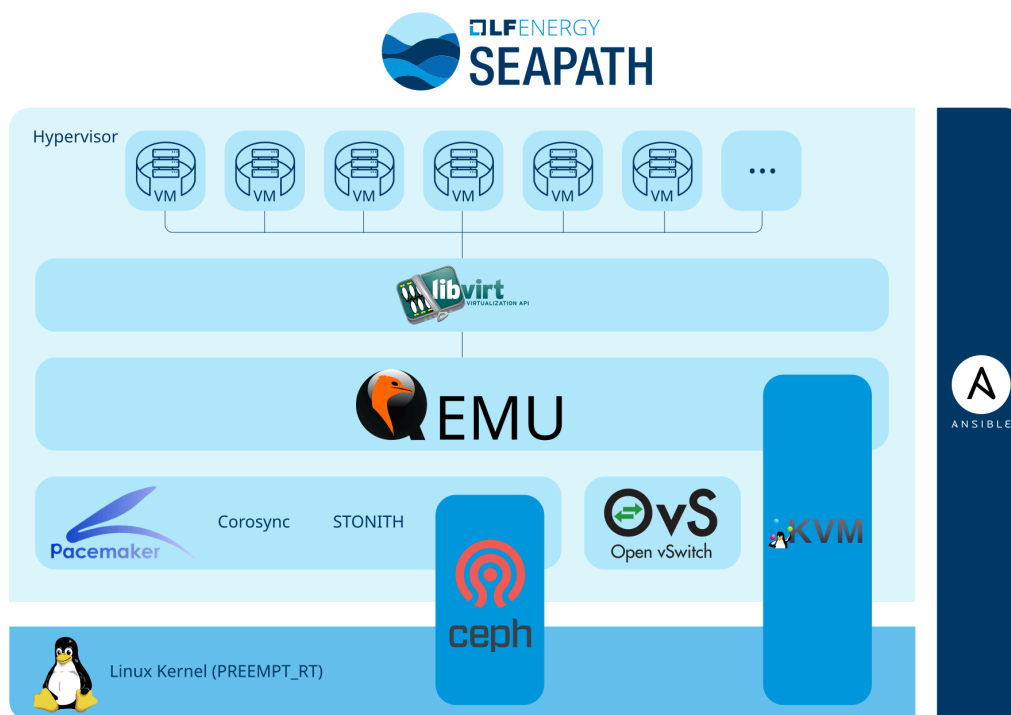


Figure 2 : Architecture générale de SEAPATH

L'**hyperviseur** tourne sur un **noyau Linux RT**, rendant préemptive la majeure partie du code du **noyau** et en particulier les sections critiques et les gestionnaires d'interruptions. Il modifie par

ailleurs certains mécanismes pour réduire les temps de latence induits par le fonctionnement du système, qui sont critiques dans notre contexte.

En plus de la couche de virtualisation décrite au Chapitre 2.2.3, SEAPATH utilise plusieurs outils permettant la mise en place d'un cluster pour avoir de la haute disponibilité :

- *Open vSwitch* [20], une implémentation logicielle d'un switch ethernet.
- *Pacemaker* [21], chargé de démarrer, arrêter et superviser les ressources du cluster.
- *Corosync* [22], un système de communication de groupe.
- *Ceph* [23], une solution de stockage distribué.

Lorsque SEAPATH est installé sur plusieurs [hyperviseurs](#), ces outils orchestrent le déploiement des [VMs](#) sur les machines adéquates et permettent de les déplacer en cas de besoin, mais surtout de les redémarrer sur une autre machine si la leur tombe en panne.

2.2.5 Génération et déploiement

Concrètement, SEAPATH se présente à la manière d'une [distribution](#) Linux : les utilisateurs disposent d'images disques pour installer SEAPATH sur leurs machines. SEAPATH se décline en plusieurs versions, chacune générant ces images de manières différentes. Nous nous concentrons sur 2 d'entre elles :

- la version «Yocto» [24] construit l'image disque *from scratch* : toute la distribution est décrite dans du code. Tous les paquets et bibliothèques nécessaires (e.g. Corosync, libvirt, etc.) possèdent leurs «recettes» (écrites dans le langage BitBake propre à Yocto) qui détaillent comment télécharger les codes sources, les construire, les empaqueter, etc.
- la version «Debian» [4] se base sur la distribution du même nom. On décrit simplement au système de build la liste des paquets à installer, quelques fichiers de configuration et on s'aide de scripts pour automatiser toute la génération.

Enfin, on utilise Ansible [25], un outil d'automatisation, pour configurer divers éléments de l'infrastructure déployée en utilisant des *playbooks*. Ces playbooks définissent l'état souhaité de l'infrastructure sous forme de code, décrivant les tâches à exécuter sur les différents nœuds du système, également appelés «hosts». Les nœuds sont décrits dans un *inventaire* qui est fourni au playbook.

2.2.6 Aspects de cybersécurité

Actuellement, le projet SEAPATH ne propose pas de moyen simple à ses utilisateurs pour suivre les vulnérabilités logicielles présentes dans le système. Les mainteneurs eux-mêmes n'ont pas connaissance de celles-ci, ce qui est un grave problème, SEAPATH étant destiné à terme à être utilisé dans des infrastructures critiques où une vulnérabilité grave peut priver d'électricité des millions de foyers et causer de graves dommages.

De plus, le monde de la cybersécurité s'accélère depuis quelques années. Les vulnérabilités découvertes sont de plus en plus nombreuses, et les organisations étatiques et même internationales commencent à légiférer pour obliger les fournisseurs de solutions numériques à se préoccuper de ces questions. Par exemple, l'Union européenne a créé le Cyber Resilience Act (CRA) [26] qui prendra effet en 2027, d'où l'intérêt pour les mainteneurs de SEAPATH d'agir maintenant. Le stage effectué s'inscrit dans ce contexte, pour permettre à SEAPATH de se doter des outils nécessaires pour suivre les vulnérabilités potentiellement dangereuses.

3 VULNÉRABILITÉS SUR SEAPATH YOCTO

La première partie du stage a été consacrée à la recherche et à l'essai des différentes possibilités pour détecter des vulnérabilités dans la version Yocto de SEAPATH. Nous n'allons pas ici analyser le code lui-même pour y trouver des failles : SEAPATH étant une [distribution](#) Linux entière, composée de centaines de composants différents, la surface à analyser est bien trop importante. Toutes les méthodes de détection étudiées prennent une autre approche : elles essaient de faire des liens entre SEAPATH et les bases de données de vulnérabilités connues. Nous allons voir comment. Une fois qu'une solution aura été sélectionnée, nous verrons sa mise en place réelle dans SEAPATH.

3.1 État de l'art

3.1.1 Solutions avec SBOM

Comme introduit dans le Chapitre 2.2.5, la construction de la [distribution](#) se fait entièrement depuis les sources : aucun binaire opaque n'est inclus dans l'image disque. Cela donne un grand avantage pour la détection des vulnérabilités car on connaît absolument tout ce qui sera inclus dans le système une fois déployé : programmes, bibliothèques, configurations, mais aussi quels fichiers sources ont été compilés, dans quelles versions, etc. Cela nous permet de facilement générer des [Software Bill of Materials \(SBOM\)](#) très complets. Yocto propose d'ailleurs plusieurs *classes* pour exporter des [SBOMs](#) dans différents formats : SPDX2, SPDX3, OpenVEX ou encore CycloneDX [27]. Un extrait de [SBOM](#) généré par Yocto pour SEAPATH est disponible en Figure 3.

Parmi les solutions trouvées lors de l'état de l'Art, plusieurs d'entre elles se basent uniquement sur ce SBOM pour trouver des vulnérabilités dans SEAPATH. C'est le cas par exemple de *Grype* [28] ou de *sbom-cve-check* [29]. Ces outils vont utiliser les champs d'identification des paquets (lignes surlignées en rouge dans la Figure 3) pour chercher les vulnérabilités associées dans des bases de données en ligne. Il en existe plusieurs :

- la NVD (National Vulnerability Database) [30], maintenue par le NIST, l'institut États-Unien des normes et de la technologie ;
- la liste du CVE™ Program [31], contenant des vulnérabilités de nombreuses autres bases de données ;
- la liste des vulnérabilités propres au [noyau](#) Linux [32], gérée par les mainteneurs du noyau ;
- les bases de données propres aux grandes distributions Linux : le traqueur de Debian [33], celui de RedHat [34], etc.

Certaines de ces bases de données associent des identifiants [Common Platform Enumeration \(CPEs\)](#) à leurs vulnérabilités : *Grype* va les faire correspondre aux [CPEs](#) présentes dans les SBOM pour dire si une vulnérabilité est présente ou non dans l'image. L'outil *sbom-cve-check* va plutôt se reposer sur les versions fournies dans le SBOM pour voir si une vulnérabilité est exploitable.

3.1.2 Solutions sans SBOM

Une autre approche pour trouver les vulnérabilités a également été étudiée : le projet Yocto propose une classe dédiée, nommée *cve-check* [35], qui va effectuer la recherche de vulnérabilités pendant le processus de construction de l'image, en bénéficiant de tout le contexte nécessaire : nom et versions des paquets, liste des fichiers sources, etc. En activant cette classe, un fichier OpenVEX est créé à la fin du build, contenant les vulnérabilités détectées.

Enfin, d'autres membres de Savoir-faire Linux ont développé des classes (au sein d'un outil nommé *meta-vulnscout* [36]) permettant d'améliorer *cve-check* pour les vulnérabilités liées au

```

1 {
2   "@context": "https://spdx.org/rdf/3.0.1/spdx-context.jsonld",
3   "@graph": [
4     {
5       "type": "software_Package",
6       "software_packageVersion": "20.2.1",
7       "name": "ceph",
8       "summary": "User space components of the Ceph file system"
9       "description": "User space components of the Ceph file system.",
10      "externalIdentifier": [{ "identifier": "pkg:yocto/meta-seapath/ceph@20.2.1"},
11        { "identifier": "cpe:2.3:*:*:ceph_storage_osd:20.2.1:*:*:*:*:*"}, ...],
12      // Identifiants du paquet aux formats standard dans l'industrie : Common Platform
13      // Enumeration (CPE) et Package-URL (PURL).
14    },
15    {
16      "type": "software_File",
17      "name": "usr/lib/ceph/erasure-code/libec_clay.so",
18      "verifiedUsing": [{ "algorithm": "sha256", "hashValue":
19        "0a323c42c25bf33d08334ac7cd2dda1b10b734f5f07714f9185c6a48db8feee4"}]
20    },
21    // Un fichier présent dans l'image disque, ici une librairie compilée.
22    {
23      "type": "software_File",
24      "software_primaryPurpose": "source",
25      "name": "sources/linux-mainline-rt-6.12.89-rt18+git/drivers/net/ipa/data/ipa_data-
26        v5.5.c",
27      "verifiedUsing": ...
28    },
29    // Un fichier source ayant servi à la compilation d'un paquet, ici un pilote inclus
30    // dans le noyau Linux.
31  ]
32 }

```

Figure 3 : Extrait d'un SBOM de SEAPATH au format SPDX3, simplifié et annoté

noyau Linux : de base, cet outil utilise uniquement sur la base de données du NIST [30] qui n'est pas complète pour le **noyau**. Ainsi, les nouvelles classes de **meta-vulnscout** vont utiliser des bases de données supplémentaires pour récupérer plus de vulnérabilités, mais aussi filtrer celles non applicables en regardant la version du **noyau** et les fichiers sources compilés ou non.

3.2 Comparaison des solutions

Après avoir établi une liste de solutions potentielles dans le Chapitre 3.1, il a fallu sélectionner celle la plus adaptée pour SEAPATH. Pour cela, chacune de ces solutions a été testée pour en tirer des évaluations quantitatives : on a mesuré le **nombre total de vulnérabilités** remontées, et on a évalué le nombre de **faux positifs** (vulnérabilités annoncées comme applicables par les outils mais qui ne le sont pas) et de **faux négatifs** (vulnérabilités non reportées par les outils).

3.2.1 Méthodologie

On a créé 4 scénarios de test qu'on a lancés sur une même version de SEAPATH pour pouvoir comparer les résultats :

- scénario «basique» utilisant uniquement la classe **cve-check** de Yocto ;
- scénario «léger» utilisant **cve-check** avec 2 des 3 classes de **meta-vulnscout** ;

- Dans ce scénario, on n'a pas inclus la classe filtrant les vulnérabilités selon les fichiers sources compilés du [noyau](#), à la demande de collègues qui voulaient estimer la performance de cette classe.
- scénario « complet » utilisant `cve-check` avec toutes les classes de `meta-vulnscout` ;
- scénario « externe » utilisant l'outil `sbom-cve-check` avec les [SBOMs](#) générés par Yocto.

On notera ici l'absence d'un scénario utilisant Grype : en effet, lors des tests préliminaires pour mettre en place ces outils, on a remarqué une quantité flagrante de faux positifs. Par conception, Grype est un outil fait pour fonctionner sur des distribution Linux « connues » : il va chercher dans les bases de données adaptées selon la distribution et se base sur les [CPEs](#), qui contiennent également des indications de distributions. SEAPATH Yocto étant une distribution « maison », Grype fonctionne moins bien. On l'a donc retiré de notre liste d'outils à tester.

3.2.2 Nombre de vulnérabilités détectées

La première chose intéressante à remarquer dans Figure 4 et Tableau 1 est le volume total de vulnérabilités détectées par les outils : en comparaison avec la classe `cve-check` seule, les classes de `meta-vulnscout` et l'outil `sbom-cve-check` ont détecté environ 2000 vulnérabilités de plus. En effet, comme expliqué précédemment, la seule base de données de `cve-check` est la NVD [30] qui n'est pas complète, en particulier concernant les vulnérabilités liées au [noyau](#) Linux. Les classes de `meta-vulnscout` utilisent les mêmes données que `cve-check` *plus* la base de données du programme CVE™ [31] et la base officielle du [noyau](#) [32]. En revanche, `sbom-cve-check` n'utilise que les bases de données NVD et CVE™ (mais cette dernière contient toutes les vulnérabilités de la base du [noyau](#)). Un résumé est disponible au Tableau 2.

Une autre chose qu'on peut remarquer dans le volume total de vulnérabilités détectées est que `sbom-cve-check` a détecté ~230 vulnérabilités de moins que `meta-vulnscout`. C'est en réalité parce que le premier filtre automatiquement celles qui ont été rejetées par les autorités de numérotation (e.g. [CVE-2021-47084](#)) tandis que le second les garde comme « non affecté », son volume total est donc naturellement plus grand.

Tableau 1 : État des vulnérabilités détectées selon les scénarios

	Corrigé	Non affecté	À évaluer	Total
basique	14 957	95	131	15 183
léger	16 534	378	363	17 275
complet	16 534	453	287	17 274
externe	16 520	403	120	17 043

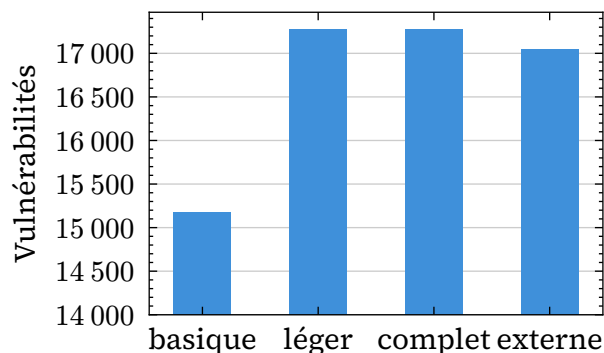
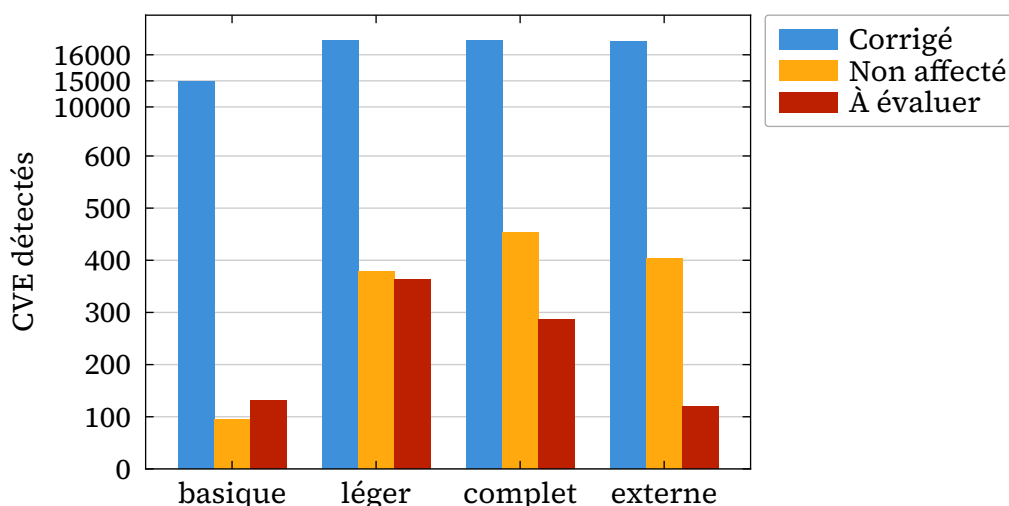


Figure 4 : Nombre total de vulnérabilités détectées par scénario.
Attention : l'axe y ne commence pas à 0.

Tableau 2 : Résumé des bases de données utilisées par les outils

	NVD	CVE™	BDD du noyau	
cve-check	✓	✗	✗	Ne détecte pas certaines vulnérabilités
meta-vulnscout	✓	✓	✓	Exporte toutes les vulnérabilités
sbom-cve-check	✓	✓	✗	N'exporte pas les vulnérabilités rejetées

**Figure 5 : États des vulnérabilités détectées par scénario de détection.**

Attention : l'axe y n'est pas linéaire pour les grandes valeurs.

3.2.3 Nombre de vulnérabilités à évaluer

Une des mesures les plus importantes est le nombre de vulnérabilités avec le statut «à évaluer» : cela nous indique combien de temps les mainteneurs et/ou utilisateurs de SEAPATH doivent consacrer à l'évaluation des vulnérabilités. Plus cette valeur est faible, mieux c'est, tant que cela n'implique pas de faux négatifs.

À ce stade, on peut déjà établir que le `cve-check` basique contient beaucoup de faux négatifs puisqu'il est basé sur une seule base de données incomplète, comme nous l'avons vu dans la section précédente. On n'utilisera donc pas ce scénario pour SEAPATH.

En ce qui concerne les scénarios `meta-vulnscout`, il est très clair que l'ajout de la classe filtrant les vulnérabilités non compilées diminue beaucoup le nombre de vulnérabilités à évaluer sans pour autant produire de faux négatifs : comme indiqué dans Tableau 1, le scénario *complet* a ~20% moins de CVE dans le statut «à évaluer» que le scénario *léger*. Et ceci est pour une version récente du `noyau` Linux, pour les versions anciennes cela peut être beaucoup plus élevé. À la demande de collègues, la même comparaison a été faite pour une version de SEAPATH datant de plus d'un an et on obtient 70% moins de CVE à investiguer (données disponibles dans le Tableau 3).

Tableau 3 : Données brutes des statuts des CVE pour SEAPATH 1.0.0

	Corrigé	Non affecté	À évaluer	Total
basique	12 524	81	2552	15 157
complet	13 925	1670	765	16 360

`sbom-cve-check` produit encore moins de vulnérabilités à évaluer. Afin de comprendre pourquoi, on a étudié finement les différences entre les résultats de chaque scénario. Il semble que la meilleure performance de `sbom-cve-check` provient d'une meilleure méthode de filtrage

sur les vulnérabilités du **noyau** non-compilés. En termes de vulnérabilités rapportées, peu importe leurs statuts, on a vérifié que `sbom-cve-check` rapporte *toutes* celles de `meta-vulnscout` avec quelques-unes en plus. Il s'agit donc du meilleur scénario, contenant moins de faux positifs et pas (à notre connaissance) de faux négatifs.

3.2.4 Choix final et compte-rendu

Ces résultats nous ont donné les clés nécessaires pour choisir la solution que l'on va utiliser dans SEAPATH : grâce à ses taux de faux positifs et négatifs moindres que les autres, l'outil `sbom-cve-check` nous a paru être le plus adapté. De plus, il possède un autre grand avantage qui est sa capacité à fonctionner en dehors du processus de build Yocto : le **SBOM** est généré lors de la construction, puis on peut relancer la détection des vulnérabilités à différents moments sans avoir à relancer la construction. Cela nous servira lorsque l'on mettra en place l'**Intégration Continue** sur SEAPATH, au Chapitre 3.5.2.

Tous ces résultats et analyses ont été consignés dans un rapport (consultable en Annexe 2) à la demande de collègues de Savoir-faire Linux. Les données de comparaison avant/après filtrage sur les vulnérabilités non compilées ont été particulièrement appréciées car elles permettent de chiffrer plus facilement les offres de gestion de vulnérabilités pour les clients de l'entreprise.

3.3 Mise en place de l'outil

Une fois la solution choisie, il a fallu l'intégrer de manière permanente dans SEAPATH. `sbom-cve-check` étant un outil externe, les seuls changements impliqués dans la base de code Yocto concernent la génération des **SBOM**. En effet, pour avoir les meilleurs résultats, `sbom-cve-check` requiert :

- un fichier SPDX contenant les contenus «classiques» générés par Yocto : liste des paquets, recettes et autres, mais également la liste des fichiers sources compilés pour le noyau Linux ;
- un fichier OpenVEX contenant les annotations pour les vulnérabilités déjà connues.

Il existe des classes pré-faites pour cela dans Yocto, ainsi il n'y a pas eu de difficulté pour écrire la configuration pour générer ces fichiers, en se basant sur la documentation du projet Yocto [24]. La Figure 6 montre le code qu'il a fallu rajouter.

BitBake

```
1 inherit create-spdx-2.2
  # Génère des fichiers SPDX pour tous les paquets inclus dans l'image générée
2 inherit vex
  # Génère un fichier OpenVEX contenant les annotations de vulnérabilités déjà connues
3
4 SPDX_INCLUDE_COMPILED_SOURCES:pn-linux-mainline-rt = "1"
  # Indique à la classe create-spdx-2.2 d'inclure la liste des sources compilées dans le
  fichier SPDX, uniquement pour le package linux-mainline-rt
```

Figure 6 : Paramètres pour générer les fichiers nécessaires à la détection de vulnérabilités

Pourtant, après essai de cette solution, la liste des sources du **noyau** n'était pas exportée dans le fichier SPDX. La documentation de Yocto n'étant pas plus détaillée sur la question, une recherche approfondie dans le code source de Yocto a été menée pour comprendre comment cette liste est générée.

On a rapidement compris que le problème provenait de la recette utilisée pour générer le **noyau** Linux. Yocto propose plusieurs recettes pré-faites pour cela afin de faciliter la création de projets avec Yocto, une recette de **noyau** étant assez complexe à écrire. Cependant, SEAPATH a divergé sur une recette maison car il y a des besoins spécifiques : comme expliqué au Cha-

pitre 2.2.4, il faut un noyau **Temps Réel** avec des configurations particulières. Les deux recettes ont divergé il y a tellement longtemps (le premier commit de celle de SEAPATH date d'octobre 2020¹) que leurs codes sources sont très différents et donc particulièrement difficiles à comparer.

Finalement, après comparaison en profondeur des fichiers générés par la construction du noyau par les deux recettes, il est apparu que le processus de build de SEAPATH ne générerait pas certains fichiers de débogage utilisés par Yocto pour trouver la liste des fichiers sources. On est ainsi arrivé à la racine du problème : il manquait deux options dans la configuration même du **noyau** Linux. Après ajout de celles-ci (cf. Figure 7), les fichiers sources sont bien listés dans le fichier SPDX généré.

```
1 # This allows to get the debug symbols of the kernel and know which files have been
  compiled. It has been pulled from https://git.yoctoproject.org/yocto-kernel-cache/tree/
  features/debug/debug-kernel.cfg?h=yocto-6.12
2 CONFIG_DEBUG_INFO_DWARF_TOOLCHAIN_DEFAULT=y
3 CONFIG_DEBUG_INFO=y
```

Figure 7 : Fichier de configuration rajouté à la recette du **noyau** Linux

3.4 Analyse des vulnérabilités détectées

Une fois les fichiers SBOM correctement générés, on a pu les faire ingérer à `sbom-cve-check` puis on a regardé les vulnérabilités détectées. Nous avons pour cela utilisé le logiciel VulnScout [3], qui est un outil d'évaluation de vulnérabilités développé par Savoir-faire Linux. Une partie significative du stage ayant porté sur l'amélioration de cet outil, nous la détaillerons dans un chapitre dédié (Chapitre 5).

Déjà, on a pu remarquer un nombre total de vulnérabilités potentiellement applicables assez importante : environ 130. Toutes les analyser aurait potentiellement pu prendre plusieurs jours voire même semaines, il a donc été décidé de les filtrer selon leur importance. On a utilisé pour cela les deux métriques courantes pour estimer l'importance de vulnérabilités : le **Common Vulnerability Scoring System (CVSS)** qui indique la sévérité et l'**Exploit Prediction Scoring System (EPSS)** qui indique l'exploitabilité. Une vulnérabilité avec un score de sévérité faible et une exploitabilité nulle ne vaut pas le coup d'être étudiée. La portée de notre analyse a donc été définie uniquement pour les vulnérabilités vérifiant une de ces conditions :

- $CVSS \geq 9.0$ (vulnérabilités critiques)
- $CVSS \geq 7.0 \wedge EPSS \geq 50\%$ (vulnérabilités importantes et facilement exploitables)

Ce choix a été fait avec le tuteur de stage car il semblait cohérent avec les vulnérabilités détectées. Nous avons consulté des collègues ayant de l'expérience dans le milieu de l'analyse de vulnérabilités, notamment dans l'équipe de Savoir-faire Linux à Montréal, pour nous assurer que les seuils étaient plausibles. Ils pourront de toute manière être réévalués, notamment lors de futures réunions du **Comité de Pilotage Technique (TSC)** de SEAPATH. L'étude des vulnérabilités étant encore au stade de R&D, il n'y a pas de problème à prendre cette décision unilatéralement pour le moment.

Ainsi, après avoir filtré la centaine de vulnérabilités détectées avec les seuils énoncés précédemment, il n'est plus resté qu'une quinzaine de vulnérabilités à analyser, dont une majorité étaient des faux positifs :

¹Commit à l'origine de la recette du noyau Linux de SEAPATH : <https://github.com/SEAPATH/meta-seapath/commit/737b85673d78e37371cff5d9589190995c03ebbf>

- Certains étaient dus à la configuration de SEAPATH. Par exemple [CVE-2026-32746](#), une vulnérabilité dans le binaire *telnetd* contenu dans le paquet *inetutils*, est détectée alors que SEAPATH contient une partie seulement d'*inetutils* (comme *ping* par exemple) mais pas *telnetd*.
- La plupart sont des vulnérabilités marquées comme potentiellement applicables alors qu'elles ont été corrigées depuis longtemps. C'est souvent le cas lorsque les bases de données de vulnérabilités n'ont pas connaissance des versions auxquelles les vulnérabilités ont été corrigées. Par exemple, [CVE-2018-6764](#) est une vulnérabilité de *libvirt* qui a été découverte et corrigée en 2018 dans la version 4.1.0. Pourtant, les bases de données n'incluent pas cette information. Il s'agit en général d'un oubli de la part des mainteneurs de ces bases, mais ça occasionne ce genre de problèmes.

Afin de corriger ces faux positifs, on peut utiliser un mécanisme de Yocto qui permet d'annoter une vulnérabilité dans une recette et qui inclut ensuite cette annotation dans le fichier OpenVEX (voir Figure 8 et Figure 9). Quand on utilise `sbom-cve-check`, on lui donne le fichier OpenVEX en plus du SBOM pour qu'il filtre automatiquement les vulnérabilités connues selon leur statut (ici, *ignored*).

BitBake

```
1 # Dans recipes-connectivity/inetutils/inetutils_.bbappend
2 CVE_STATUS[CVE-2026-32746] = "not-applicable-config: telnetd not included in SEAPATH"
3
4 # Dans recipes-extended/libvirt/libvirt_.bbappend
5 CVE_STATUS[CVE-2018-6764] = "fixed-version: Fixed in 4.1.0, NVD tracks this as version-
  less vulnerability"
```

Figure 8 : Annotations de vulnérabilités dans les recettes BitBake

```
1 {
2   "name": "inetutils",
3   "version": "2.5",
4   "cpes": ["cpe:2.3:*:*:inetutils:2.5:*:*:*:*:*"],
5   "issue": [{
6     "id": "CVE-2026-32746",
7     "status": "Ignored",
8     "link": "https://nvd.nist.gov/vuln/detail/CVE-2026-32746",
9     "detail": "not-applicable-config",
10    "description": "telnetd not included in SEAPATH"
11  }, ...]
12 }
```

Figure 9 : Extrait de fichier OpenVEX contenant une annotation de vulnérabilité

Pour les faux positifs causés par l'absence du numéro de la version corrigeant la vulnérabilité, il a fallu trouver ce numéro. C'était parfois trivial lorsque le correctif est annoncé dans les notes de versions, mais il a souvent fallu fouiller les dépôts Git à la recherche des commits contenant les correctifs pour ensuite trouver les versions associées.

Afin que ce travail bénéficie à d'autres utilisateurs de Yocto, ces annotations ont été rendues *open source* : après relecture par plusieurs collègues, le commit a été soumis aux mainteneurs du projet Yocto via leur liste de diffusion². Il a ensuite été accepté quelques semaines plus tard, ces changements font donc officiellement partie de la base de code publique du projet Yocto.³

²Discussion sur la liste de diffusion : <https://lists.yoctoproject.org/g/meta-virtualization/topic/118343262>

³Commit contenant les changements : <https://git.yoctoproject.org/meta-virtualization/commit/?id=88e29d1f7f>

Concernant les vrais positifs (les vulnérabilités détectées qui sont réellement présentes et potentiellement exploitables), il n'est pas dans le cadre du stage de les corriger. SEAPATH étant un projet encore jeune, il n'est pas encore déployé dans de vraies infrastructures critiques. De plus, SEAPATH est open-source et les mainteneurs n'ont pas encore décidé de la politique qu'ils comptaient adopter concernant les vulnérabilités détectées en son sein.

Enfin, la documentation de SEAPATH Yocto a été mise à jour pour indiquer aux futurs contributeurs et utilisateurs comment effectuer cette analyse de vulnérabilités, en listant les fichiers à utiliser et les commandes à exécuter pour les traiter.

3.5 Intégration Continue

Pour le moment, la détection des vulnérabilités a été faite manuellement après chaque build de SEAPATH. La prochaine étape est d'intégrer cette détection dans le pipeline de [CI](#). Grâce à cela, à chaque changement dans le code de SEAPATH, le système va chercher les vulnérabilités et générer un rapport. C'est particulièrement pratique pour les [Pull Request \(PR\)](#) : cela permettra de détecter aisément si un contributeur externe essaie d'introduire une faille dans SEAPATH car les vulnérabilités ajoutées apparaîtraient automatiquement dans le résumé de la [PR](#).

Jusqu'à maintenant, la [CI](#) de SEAPATH est interne à Savoir-faire Linux : elle tourne sur une instance Jenkins [10] et est lancée dès qu'un commit est prêt à être intégré dans la base de code Gerrit [8] interne à l'entreprise. Cependant, SEAPATH est un projet [open source](#) dont les dépôts publics sont aussi hébergés sur GitHub [9]. Les personnes externes à Savoir-faire Linux qui souhaitent contribuer à SEAPATH le font sur GitHub et ne déclenchent donc pas la [CI](#), ce qui nullifie en partie l'intérêt d'ajouter la détection de vulnérabilités dans la [CI](#). De plus, puisque Jenkins est uniquement accessible en interne chez Savoir-faire Linux, il n'y a pas de transparence sur les processus utilisés lors du développement. Pour ces raisons, il a été décidé en premier lieu de réécrire la [CI](#) sur GitHub Actions [11] pour ensuite y rajouter la détection de vulnérabilités.

3.5.1 Transfert sur GitHub Actions

Le pipeline de [CI](#) de SEAPATH sur Jenkins est assez basique : il récupère les sources, initialise l'environnement de développement (à l'aide de CQFD, voir Chapitre 2.1.2) puis lance successivement la construction de 4 variantes d'images SEAPATH différentes (pour chaque type d'image utilisé dans un cluster). Si tout s'est passé correctement, la CI passe. Si un problème survient, la CI s'arrête instantanément.

Il est très aisé de reproduire une [CI](#) aussi basique sur GitHub Actions : les actions exécutées n'ont rien de particulier, notre pipeline `build` ne contient presque qu'une suite de commandes shell à exécuter.

Une première amélioration menée a été de diviser le processus de build : au lieu de construire toutes les variantes dans une seule et même étape du pipeline (voir Figure 10a), on a utilisé la fonctionnalité de *matrice* de GitHub Actions permettant d'effectuer une même suite d'étapes plusieurs fois avec des paramètres différents : ici, on itère sur la variante d'image à construire (voir Figure 10b). Cela a de nombreux avantages : on peut voir les journaux de chaque étape individuellement, en cas de problème lors de la construction d'une variante la CI continuera pour les autres, on peut même paralléliser la construction pour la rendre plus rapide.

Une fois le pipeline écrit, on a rencontré un premier problème : un build de SEAPATH « à froid », ce qui veut dire sans jamais ne l'avoir fait auparavant et sans avoir mis en cache des données, peut durer plusieurs heures et demande des ressources non négligeables en mémoire et processeur.

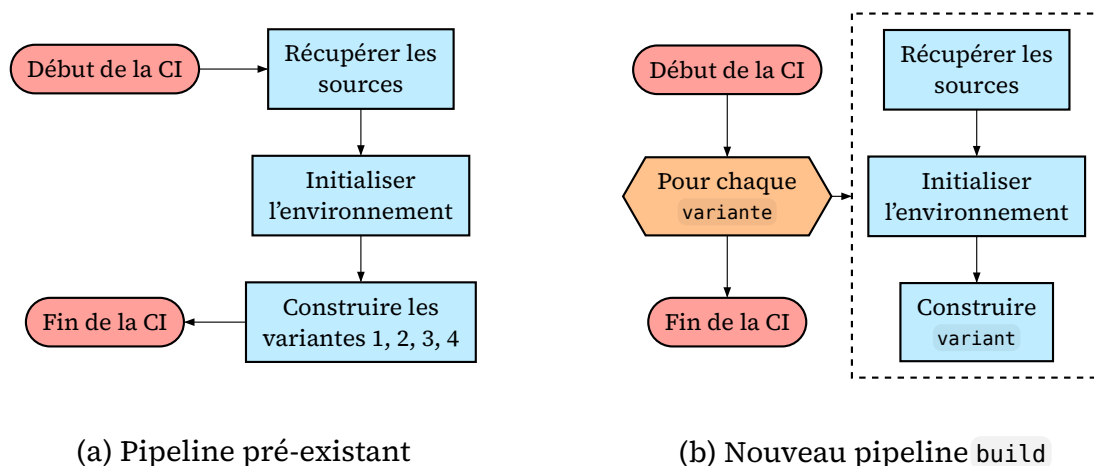


Figure 10 : Évolution de l'architecture du pipeline de construction des images

Il est irréaliste de faire tourner la **CI** sur l'infrastructure publique de GitHub. Un *runner* GitHub Actions a donc été déployé sur un serveur dédié de l'entreprise : on bénéficie ainsi d'une plus grande puissance de calcul et d'un cache stocké localement. Une fois passé la toute première construction, les exécutions de CI suivantes prennent une vingtaine de minutes seulement.

3.5.2 Détection de vulnérabilités

Maintenant que les images se construisent correctement dans la **CI**, la détection des vulnérabilités sur les images générées a pu être automatisée. Nous avons donc écrit un nouveau pipeline dédié qui s'exécute après celui de build.

Comme décidé dans le Chapitre 3.2.4, on va utiliser l'outil *sbom-cve-check* qui utilise les **SBOMs** générés par l'étape de construction de l'image. Le pipeline commence par installer cet outil. Il télécharge ensuite les **SBOMs** de chaque variante, puis lance la détection de vulnérabilités sur ceux-ci. On a bien fait attention de faire la détection sur chaque fichier individuellement : cela permettra de savoir dans quelle(s) variante(s) les vulnérabilités sont présentes. *sbom-cve-check* exporte un fichier SPDX3 par variante, contenant le **SBOM** ainsi que les vulnérabilités détectées.

Ensuite, nous utilisons l'outil VulnScout pour générer des rapports sur les vulnérabilités présentes dans les fichiers SPDX3 ainsi que pour émettre une erreur lorsque des vulnérabilités dépassent les seuils définis au Chapitre 3.4. Les rapports sont ensuite sauvegardés et rendus publics à la fin du pipeline pour pouvoir être lus par n'importe qui.

En pratique, nous exportons un fichier CSV (un tableur) contenant l'ensemble des vulnérabilités détectées avec leurs statuts, ainsi qu'un résumé permettant aux utilisateurs d'avoir une vue rapide des vulnérabilités présentes. Un exemple peut être consulté en Annexe 4.

Lorsque des vulnérabilités dépassent les seuils, on a pris soin de rendre l'erreur la plus claire possible pour que les mainteneurs puissent avoir l'information rapidement et facilement. Ainsi, une liste des vulnérabilités critiques et leurs scores **EPSS** et **CVSS** ainsi que les packages et variantes associés, sont affichées en annotation sur la page du pipeline (voir Figure 11).

Comme indiqué dans le Tableau 2 du Chapitre 3.2.2, *sbom-cve-check* a besoin de télécharger les bases de données du NVD et du projet CVE™. Celles-ci sont particulièrement conséquentes : elles représentent à elles deux 7,1 Go de données. Il n'est pas raisonnable de télécharger ces bases de données à chaque fois que la CI tourne. Ainsi, nous faisons aussi tourner la CI sur le serveur

Summary

All jobs

Prepare

Call CVE check workflow

CVE check

Run details

Usage

Workflow file

Annotations

1 error and 1 warning

Call CVE check workflow / CVE check

Process completed with exit code 2.

VulnScout

Some vulnerabilities triggered fail condition:

Total failed vulnerabilities: 5

CVE-2026-33845

Severity: CRITICAL

CVSS Score: 9.1

EPSS Score: 0.06%

Affected Packages:

- gnutls (3.8.12)

Affected Variants:

- host_standalone_efi

- guest_efi

CVE-2026-42010

Severity: CRITICAL

...

Figure 11 : Page de résumé du pipeline de détection des vulnérabilités.

dédié de Savoir-faire Linux, en configurant notre pipeline pour que les bases de données soient stockées dans un emplacement qui ne soit pas régulièrement supprimé.

Enfin, puisque des vulnérabilités sont découvertes tous les jours, on a mis en place un pipeline `periodic-cve-check` qui s'exécute automatiquement tous les matins et va détecter les vulnérabilités sur la dernière image de SEAPATH construite en utilisant le [SBOM](#) sauvegardé du dernier pipeline. Ainsi, même si il n'y a pas eu d'activité sur le dépôt, la détection se fait tout de même régulièrement et permet de ne pas laisser une nouvelle vulnérabilité critique non détectée.

3.5.3 Pull Requests

Jusqu'ici, le pipeline de construction ne s'exécutait que lorsqu'un commit était poussé sur la branche principale du dépôt. Cependant, la politique de contribution à SEAPATH impose de pousser son code sur une branche à part dans un dépôt personnel, puis d'ensuite faire une [Pull Request](#) sur le dépôt principal. Nous avons donc créé un pipeline qui se lance lorsque qu'une [PR](#) est ouverte vers la branche principale : celle-ci va construire les images, puis faire la détection de vulnérabilités.

Afin de ne pas dupliquer de code, nous avons pris avantage du concept des pipelines réutilisables de GitHub Actions [37] : cela permet d'«inclure» un pipeline dans un autre. Ainsi, nous avons créé 2 pipelines réutilisables, qui sont inclus dans les pipelines `push`, `pr` et `periodic-cve-check` :

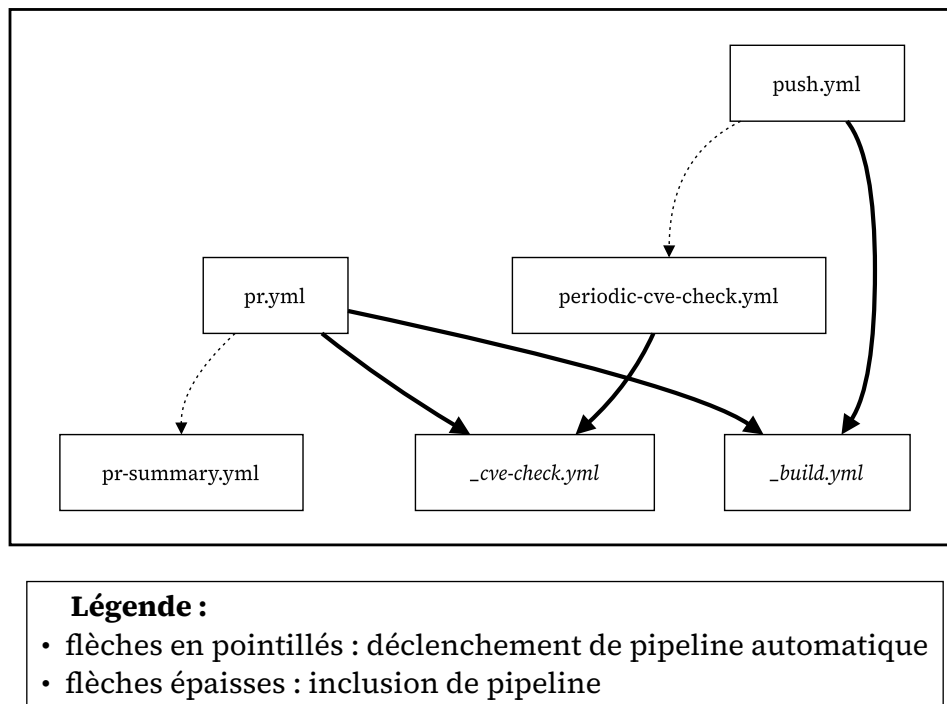


Figure 12 : Schéma de la CI

- `_build`, décrit au Chapitre 3.5.1, qui construit les images et sauvegarde les **SBOMs** ;
- `_cve-check`, décrit au Chapitre 3.5.2, qui effectue la détection des vulnérabilités sur les **SBOMs** sauvegardés précédemment et exporte des rapports.

Un diagramme montrant les différents pipelines ainsi que leurs relations est disponible en Figure 12.

De plus, afin de rendre les potentiels problèmes rencontrés lors de la **CI** les plus clairs possibles, nous avons créé un pipeline à part, `pr-summary`, qui compile les résultats et poste un commentaire sur la **PR** qui a déclenché le pipeline : ce commentaire indique si la construction des images s'est bien passée et si des vulnérabilités ont dépassé les seuils. Un exemple est disponible en Figure 13.

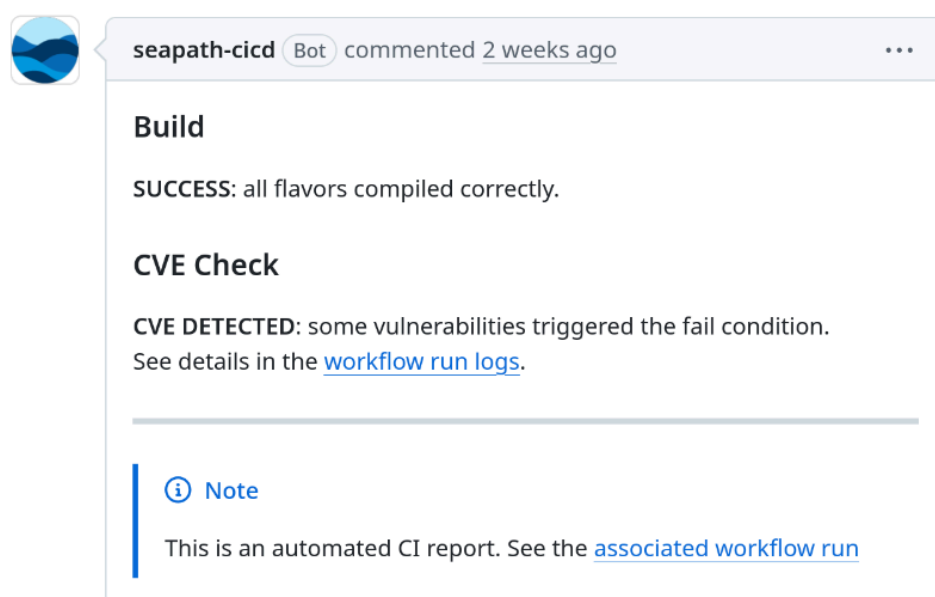


Figure 13 : Commentaire créé automatiquement sur une **PR** par la **CI**.

3.5.4 Approche multi-dépôts

Pour générer une image SEAPATH, il y a besoin de beaucoup de code : la description de l'environnement, les scripts, le code BitBake, ses dépendances, les fichiers de configurations, etc.

- L'environnement de travail, les scripts et autres fichiers de configuration sont stockés à la racine du répertoire de travail et sauvegardés dans le dépôt *yocto-bsp* [38].
- Le code BitBake décrivant la distribution à générer est stocké dans le répertoire *sources/meta-seapath* et sauvegardé dans le dépôt *meta-seapath* [39].
- Les dépendances BitBake sont tirées depuis d'autres dépôts Git qui ne sont pas maintenus par SEAPATH et sont également stockés dans le répertoire *sources/*.

Afin d'éviter aux développeurs de SEAPATH une longue suite de commandes `git clone` à chaque fois qu'il faut préparer un environnement de travail, l'outil *repo* [40] est utilisé : il permet de décrire dans un fichier *manifest* la liste des dépôts à tirer et où les placer dans l'espace de travail. Les *manifests* de SEAPATH sont stockés dans le dépôt *repo-manifest* [41]. L'arborescence créée par *repo* sur le *manifest* de SEAPATH est montrée dans la Figure 14.

Cette approche est standard dans les projets basés sur Yocto, mais elle complique la mise en place de CI. En effet, les CI basées sur GitHub Actions sont intimement liées à un dépôt : le code des pipelines est stocké dedans et tous les déclencheurs sont liés au dépôt. Cependant nous voulons qu'un commit ou une PR sur n'importe lequel des dépôts de code de SEAPATH (*yocto-bsp* ou *meta-seapath*) déclenche notre CI. Il a donc fallu trouver des solutions pour contourner le problème. Nous avons envisagé deux approche :

1. dupliquer les pipelines sur les deux dépôts ;
2. avoir les pipelines principaux sur un dépôt et leur rediriger les événements de l'autre dépôt en utilisant d'autres pipelines dédiés.

La première solution est la plus simple, mais elle a plusieurs inconvénients :

- Elle implique de dupliquer du code, ce qui induit un coût de maintenance supplémentaire et de potentiels bugs si les pipelines sont mis à jour dans un dépôt mais pas dans l'autre.

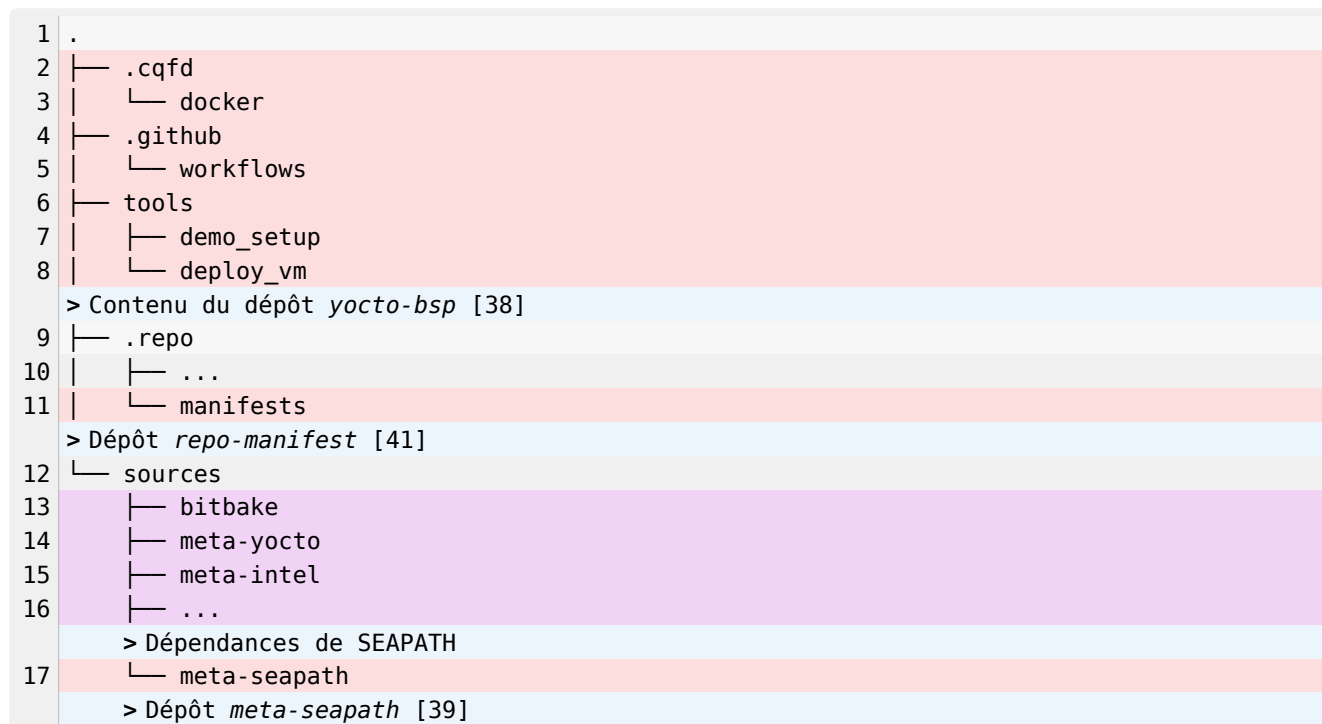


Figure 14 : Arborescence des sources de SEAPATH Yocto

- Elle disperse les résultats des exécutions de la CI : si on veut avoir une vue globale de la CI de SEAPATH Yocto, il faut ouvrir les deux dépôts et y chercher les dernières exécutions.

La deuxième solution règle ces inconvénients : les pipelines de constructions sont présent uniquement dans le dépôt principal, tandis que l'autre dépôt contient simplement des pipelines de redirection qui ne changeront jamais. De plus, tous les résultats de CI seront centralisés dans le dépôt principal. Nous avons donc choisi cette solution.

3.5.5 Synchronisation entre dépôts

Pour mettre en œuvre cette synchronisation, on va conserver les pipelines principaux (`build` et `cve-check`) dans le dépôt *yocto-bsp*. Les pipelines dans *meta-seapath* serviront uniquement à déclencher ces derniers quand un commit ou une PR arrive.

Une première difficulté a été de trouver le mécanisme à utiliser pour déclencher les pipelines du dépôt principal depuis un autre dépôt. Nous avons cherché dans la documentation s'il existait des *triggers* (les types d'événements qui déclenchent un pipeline) qui correspondraient à notre usage, et il y en avait deux :

1. `workflow_dispatch`, qui permet à des utilisateurs de manuellement lancer un pipeline avec des paramètres ;
2. `repository_dispatch`, qui est fait pour qu'un événement externe au dépôt déclenche des actions via l'[Application Programming Interface \(API\)](#) de GitHub, en y passant des paramètres.

À première vue, le deuxième *trigger* paraît mieux indiqué. Cependant, il possède deux limitations bloquantes :

- Il est impossible de suivre l'avancement d'un pipeline déclenché ainsi⁴ : à cause de cela, les pipelines dans *meta-seapath* renverront toujours « succès », même si le pipeline déclenché dans *yocto-bsp* échoue.
- On ne peut pas dire à GitHub dans quelle branche lancer le pipeline, ce qui est dommage car SEAPATH est maintenu dans différentes versions sur des branches dédiées.

Nous avons donc fait le choix du premier *trigger* : on utilise l'[API](#) de GitHub Actions pour manuellement déclencher le pipeline avec `workflow_dispatch` et on attend ensuite la fin de son exécution, ce qui n'était pas possible avec `repository_dispatch`. L'inconvénient de cette méthode est qu'elle fait apparaître un menu permettant de lancer la CI depuis l'interface utilisateur (voir Figure 15). Ce menu ne s'affichant qu'aux mainteneurs de SEAPATH, nous avons considéré qu'il s'agissait d'un problème mineur.

⁴Discussion à propos du problème de suivi des pipelines avec `repository_dispatch` : <https://github.com/peter-evans/repository-dispatch/issues/260>

Figure 15 : Élément d'interface utilisateur qui apparaît à cause du `trigger workflow_dispatch`

La deuxième difficulté de cette CI synchronisée entre 2 dépôts a été la gestion des permissions et secrets. En effet, afin de pouvoir déclencher la CI dans le dépôt principal, le pipeline de «délégation» a besoin d'un accès **authentifié** à l'API GitHub Actions. Il a donc été nécessaire de créer une «GitHub App» qui possède les permissions pour déclencher des pipelines et stocker sa clé privée dans le dictionnaire de valeurs secrètes du dépôt *meta-seapath*.

Tout ceci fonctionne très bien, sauf dans un cas très particulier : lorsqu'un contributeur externe à l'organisation SEAPATH propose une **PR**, le pipeline exécuté dispose de permissions restreintes et n'a pas accès aux secrets du dépôt. Il s'agit d'une mesure préventive mise en place par GitHub pour éviter que des **PR** malveillantes volent des données confidentielles. Dans notre cas, il n'y a pas de risques car le code de la **PR** n'est pas téléchargé dans le pipeline de délégation. Cependant, à cause de ces restrictions, il n'est pas possible de faire la délégation dans ce pipeline, puisqu'il y a besoin du secret. Il y avait là encore 2 solutions possibles :

1. Utiliser le `trigger pull_request_target` qui s'exécute sans les restrictions de sécurité ;
2. Utiliser le `trigger workflow_run` qui permet d'automatiquement lancer un pipeline à la fin d'un autre, cette fois-ci sans les restrictions de sécurité.

Ces deux options étant similaires en apparence, on a sélectionné la première car elle était plus simple à mettre en place. Le problème de permissions s'est résolu, mais un autre est apparu : la **CI** se lance automatiquement lorsqu'une **PR** est ouverte, sans que les mainteneurs n'aient à l'approuver préalablement. C'est une grave faille de sécurité : n'importe quel attaquant peut introduire des changements ayant pour but d'exécuter du code malveillant dans l'environnement de **CI** et potentiellement de corrompre la machine sur laquelle elle tourne, qui est hébergée au sein de Savoir-faire Linux (cf Chapitre 3.5.1). Malgré nos recherches, il n'existe pas de moyen simple de contourner ce problème.

Ainsi, nous avons bifurqué vers la seconde option, qui est plus complexe car elle demande un pipeline supplémentaire et quelques manipulations pour lui passer les données nécessaires. On retrouve bien le bouton pour approuver le lancement de la **CI** (voir Figure 16) qui ajoute une protection contre les attaques. La structure de la **CI** est ainsi complète. Un schéma est disponible en Figure 17.



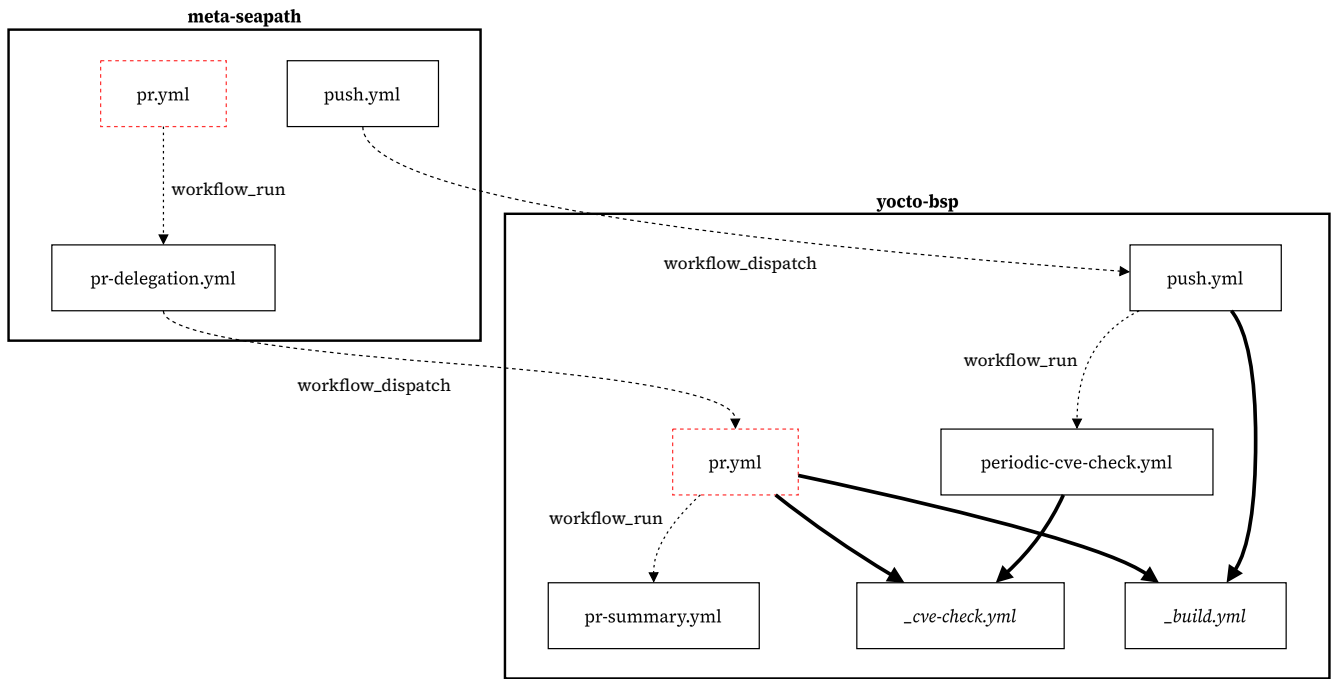
1 workflow awaiting approval

This workflow requires approval from a maintainer: [Learn more about approving workflows.](#)

Approve workflows to run



Figure 16 : Encart demandant aux mainteneurs d'approuver le lancement de la CI pour une PR venant d'un contributeur externe.



Légende :

- flèches en pointillés : déclenchement de pipeline automatique
- flèches épaisses : inclusion de pipeline
- rectangles : fichiers de pipelines
- rectangles en pointillés rouges : pipelines à permissions restreintes

Figure 17 : Schéma complet de la CI.

4 VULNÉRABILITÉS SUR SEAPATH DEBIAN

Comme indiqué dans le Chapitre 2.2.5, SEAPATH est décliné en plusieurs versions et leurs images sont complètement différentes. Ainsi, il ne suffit pas d'analyser les vulnérabilités sur la version Yocto : il faut aussi faire les autres. Dans le cadre de ce stage, nous avons également mis en place la détection des vulnérabilités sur la version Debian. Cette version étant très différente de Yocto, il a fallu utiliser d'autres approches.

4.1 Approches de détection

La version Debian de SEAPATH utilise l'outil *FAI* (Fully Automatic Installation) [42] pour construire les images. Cet outil permet de générer des supports d'installations Linux à partir de distributions existantes, contenant des paquets préinstallés et des scripts pour préconfigurer le système. En pratique, SEAPATH s'en sert pour générer des images préinstallées contenant tous les paquets nécessaires (e.g. *ceph*, *corosync* et autres pour les images cluster, voir Chapitre 2.2.4), avec le partitionnement du disque pré-établi, les fichiers de configuration des paquets déjà copiés et plusieurs scripts à lancer lors de l'installation de l'image sur une machine physique (par exemple pour automatiquement paramétrer les utilisateurs ou bien pour configurer le réseau).

Cette méthode de génération d'image est très pratique car bien plus rapide à écrire qu'une image Yocto : il suffit de décrire la liste des paquets, mettre les fichiers de configuration et écrire quelques scripts Bash, là où avec Yocto il faut complètement décrire comment compiler chaque paquet, comment copier les fichiers, etc. C'est également beaucoup plus rapide à générer car les paquets sont déjà compilés, il faut juste les télécharger depuis les archives des paquets Debian : une image peut être générée en 5 minutes, là où avec Yocto cela peut mettre jusqu'à 4 heures.

Cependant, avec cette manière de faire il n'est pas possible de générer de **SBOM** à partir des sources (comme avec Yocto) puisqu'on ne compile pas les paquets. Une approche différente a donc été utilisée en scannant l'image déjà construite. Pour cela, le logiciel **open source Syft** [43] est particulièrement adapté. C'est un outil de génération de **SBOM** à partir de plusieurs types de sources : images de **conteneurs**, fichiers binaires, archives, ou même répertoires complets. C'est ce dernier type de source qui nous intéresse : quand on lance Syft sur la racine d'un système de fichier (usuellement nommé `/`), Syft va analyser tout le système et créer un **SBOM** complet. Ainsi, en utilisant Syft sur la racine d'une image SEAPATH Debian, Syft peut générer un **SBOM** contenant tous les composants du système et on pourra détecter les vulnérabilités à l'aide de ce fichier.

4.2 Intégration du générateur de SBOM

Il a fallu décider d'où mettre en place Syft dans le processus de construction de l'image. La première solution est tout simplement de déployer une image construite dans une **VM**, d'exécuter Syft à l'intérieur puis de récupérer le **SBOM** ainsi généré. La mise en place est très simple techniquement, mais elle demande du temps et des ressources pour le déploiement de la **VM**. Une seconde solution est de s'insérer dans le processus de génération de l'image, au moment où le système de fichier est finalisé mais avant qu'il ne soit compacté en une image disque, pour lancer Syft dessus. C'est une approche plus complexe car elle demande de comprendre comment fonctionne *FAI* pour pouvoir rajouter du code au bon moment dans le processus. Cependant elle aura un temps d'exécution bien moindre que la première solution. C'est cette voie qui a été choisie.

Le processus de génération de l'image est décrit dans la Figure 18 : un script s'occupe de préparer l'environnement, de lancer FAI puis de récupérer les images. De son côté, FAI gère ses tâches

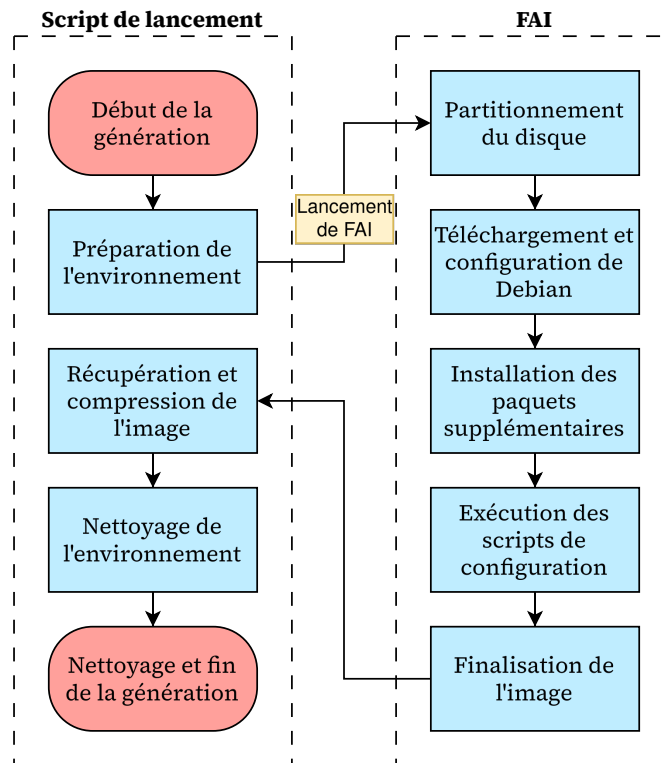


Figure 18 : Diagramme de l'exécution de la génération d'image avec FAI

avec un ordonnancement pré-défini : d'abord la configuration des partitions, puis l'installation des paquets, puis les scripts de configuration. Le moment idéal pour scanner le système de fichiers et générer un **SBOM** est à la fin de toutes les tâches mais juste avant la finalisation de l'image. Pour ce faire, on a pris avantage d'une fonctionnalité avancée de FAI : les *hooks*, qui sont des scripts bash que l'on peut insérer avant n'importe quelles tâches (les blocs bleus de la Figure 18). Un *hook* a donc été créé : il s'exécutera juste avant la tâche de finalisation pour lancer Syft. Son code simplifié est disponible en Figure 19.

bash

```

1 #!/bin/bash
2 sbom_dir="/ext/output/sbom"
3
4 mkdir -p "$sbom_dir"
5
6 function scan_rootfs {
7     echo "Scanning rootfs at $FAI_ROOT..."
8     syft scan "$FAI_ROOT" \
9         # FAI_ROOT est une variable contenant le chemin vers le système de fichier construit.
10         --source-name "SEAPATH Debian" \
11         --source-supplier "SEAPATH using FAI v$FAI_VERSION" \
12         --source-version "$SEAPATH_VERSION" \
13         -o json=/ext/output/sbom/rootfs.syft.json
14 }
15 scan_rootfs

```

Figure 19 : Hook de génération de **SBOM** avant la tâche de finalisation

Il a aussi fallu installer le binaire de Syft dans l'environnement de construction de l'image afin que le hook puisse l'utiliser. Enfin, le script de lancement a été modifié pour récupérer le **SBOM** construit à la fin et le stocker à un endroit adéquat.

4.3 Détection des vulnérabilités

Maintenant que l'on peut générer des **SBOMs**, on a pu passer à l'étape de détection des vulnérabilités. Pour ce faire, 2 possibilités ont été identifiées : réutiliser *sbom-cve-check* [29], comme ça a été fait dans la version Yocto au Chapitre 3, ou bien utiliser *Grype* [28], un outil **open source** complémentaire à Syft qui est développé par la même entreprise, Anchore.

Afin de ne pas avoir à refaire le travail de compréhension et de mise en place d'un nouvel outil, nous avons commencé par essayer d'utiliser *sbom-cve-check* sur le **SBOM** généré par Syft. Cependant, *sbom-cve-check* ne prend en entrée que des **SBOMs** au format SPDX3 ou bien au format SPDX2 spécifiquement généré par Yocto. Syft peut exporter dans plusieurs formats, mais ne supporte pas le SPDX 3. Ainsi, il n'est pas possible de directement importer les résultats de Syft dans *sbom-cve-check*. Nous avons essayé d'utiliser des convertisseurs de **SBOMs** (par exemple, SPDX2 vers SPDX3), mais ces outils induisent une perte d'information qui réduit la qualité de la détection de vulnérabilités en plus d'introduire plus de complexité dans le processus.

Afin de rester simple, nous avons donc essayé l'outil *Grype*. Celui-ci peut directement utiliser le format de sortie natif de Syft et donc dispose des informations les plus complètes possibles. Le résultat était très probant, *Grype* nous génère une liste des vulnérabilités présentes très complète (voir Figure 20) : elle contient à la fois beaucoup de vulnérabilités mais aussi beaucoup d'informations (versions des paquets corrigées, type de paquet, etc.).

On a pu expliquer ces bons résultats par les informations supplémentaires contenues dans le **SBOM** généré par Syft : celui-ci contient le type de distribution, ce qui permet à *Grype* d'utiliser des données contenues dans les bases spécifiques (dans notre cas, le Debian Security Tracker [33]) : grâce à cela, *Grype* peut afficher par exemple quand une vulnérabilité ne sera jamais corrigée (« won't fix » dans le résumé de la Figure 20).

NAME	INSTALLED	FIXED IN	TYPE	VULNERABILITY	SEVERITY	EPSS
linux-kernel	6.12.90+deb13.1-rt-amd64	*6.18.29, 7.0.6	linux-kernel	CVE-2026-43500	High	43.5%
openssh-client	1:10.0p1-7+deb13u4		deb	CVE-2020-15778	Negligible	64.3%
openssh-server	1:10.0p1-7+deb13u4		deb	CVE-2020-15778	Negligible	64.3%
openssh-sftp-server	1:10.0p1-7+deb13u4		deb	CVE-2020-15778	Negligible	64.3%
libsubid5	1:4.17.4-2	(won't fix)	deb	CVE-2024-56433	Low	6.0%
login.defs	1:4.17.4-2	(won't fix)	deb	CVE-2024-56433	Low	6.0%
linux-kernel	6.12.90+deb13.1-rt-amd64		linux-kernel	CVE-2022-3646	Medium	0.1%
stdlib	go1.24.4	1.23.12, *1.24.6	go-module	CVE-2025-47907	High	< 0.1%
...						

Figure 20 : Extrait du résumé de l'exécution de Grype

Afin d'estimer si les vulnérabilités remontées étaient cohérentes, nous avons utilisé VulnScout qui est capable d'interpréter les fichiers de résultat de *Grype* via son outil de scan intégré. Nous avons pris avantage de cette capacité en faisant un léger changement dans le code (voir Chapitre 5.1), ce qui nous a permis de voir toutes les vulnérabilités dans VulnScout. Nous avons survolé les plus importantes, et il ne semble pas y avoir de faux positifs : comme indiqué plus haut, *Grype* se sert directement du traqueur de sécurité de Debian, donc les données sont très précises.

Tout comme pour SEAPATH Yocto, il n'est pas dans le cadre du stage de corriger les vulnérabilités détectées, pour les mêmes raisons évoquées à la fin du Chapitre 3.4. Nous en sommes donc restés là pour l'analyse des vulnérabilités.

4.4 Intégration en CI

De la même manière que pour SEAPATH Yocto (Chapitre 3.5), il a fallu intégrer notre processus de détection des vulnérabilités dans l'[Intégration Continue](#). Il existait déjà un pipeline de construction des images qui appelle le script évoqué dans le Chapitre 4.2 : comme nous avons déjà modifié ce script pour récupérer les [SBOMs](#) générés, il ne reste plus qu'à lancer Grype pour détecter les vulnérabilités et VulnScout pour générer les rapports.

Le lancement de Grype se fait en quelques lignes de commandes tandis que celui de VulnScout est presque identique à ce qui a été fait sur Yocto (Chapitre 3.5.2). La subtilité est dans la gestion du cache de Grype : en effet, l'outil télécharge toutes ses données avant de faire la détection. Cela peut prendre plusieurs minutes lorsqu'on part de rien. Afin de ne pas prendre ce temps à chaque lancement de la [CI](#), mais aussi pour éviter de surcharger inutilement les serveurs de Grype, nous avons fait en sorte de conserver la base de données entre les exécutions de la [CI](#). Cependant, pour ce pipeline, nous n'utilisons pas de machine dédiée mais les *runners* publiques de GitHub Actions. Conserver des données entre les exécutions de la [CI](#) n'est plus aussi simple que de les stocker dans un répertoire spécifique, il faut utiliser le système de cache spécifique à GitHub Actions, en prenant garde à ne sauvegarder le cache que si c'est nécessaire, donc uniquement s'il y a eu des mises à jour dans la base de données de Grype. Une version simplifiée des étapes du pipeline responsable du cache est présentée en fig Figure 21.

De plus, tout comme sur SEAPATH Yocto, nous avons aussi ajouté un pipeline qui s'exécute automatiquement tous les jours pour détecter les nouvelles vulnérabilités, avec le même principe qu'expliqué dans le Chapitre 3.5.2 : on ne reconstruit pas l'image à chaque fois, on va juste réutiliser le [SBOM](#) du dernier pipeline de construction.

La [Pull Request](#)⁵ n'a pas encore été intégrée à la base de code car il reste des questions ouvertes sur la gestion des vulnérabilités détectées, notamment quoi faire des vulnérabilités que les mainteneurs de Debian ont marqué comme «won't fix». On aurait pu simplement les ignorer dans Grype, mais cela nous a semblé être une solution un peu rapide et potentiellement problématique en terme de transparence. Ces questions seront discutées lors d'une réunion du [Comité de Pilotage Technique](#).

4.5 Images de conteneurs

Une des évolutions prévues pour une future version de SEAPATH est l'ajout du support des [conteneurs](#), entre autres pour pouvoir utiliser *cephadm*, un outil permettant d'administrer des clusters Ceph [23] qui repose sur l'utilisation de conteneurs. En pratique, lorsqu'on veut lancer un conteneur avec, par exemple, Podman [44], il suffit d'une simple commande `podman run` : celle-ci va s'occuper de télécharger l'image voulue depuis Internet si elle n'est pas déjà présente dans le registre local, puis elle va créer et lancer le conteneur. Cependant, un cluster SEAPATH est en général isolé d'Internet pour des questions de sécurité. Ainsi, il n'est pas possible de télécharger l'image d'un conteneur (voir l'erreur dans la Figure 22).

```
$ podman run --rm quay.io/ceph/ceph:v20.2.0
Trying to pull quay.io/ceph/ceph:v20.2.0...
Error: unable to copy from source docker://quay.io/ceph/ceph:v20.2.0: initializing source
docker://quay.io/ceph/ceph:v20.2.0: ping container registry quay.io: Get "https://quay.io/v2/": dial tcp: lookup quay.io: Temporary failure in name resolution
```

Figure 22 : Erreur lors du lancement d'un conteneur sans connexion Internet

⁵PR ajoutant le pipeline de détection de vulnérabilités : https://github.com/seapath/build_debian_iso/pull/164

```

1 - name: Restore Grype cache
2   id: grype-cache
3   uses: actions/cache/restore@v5
4   with:
5     key: grype
6     path: "/cache/grype"
7     # Nous téléchargeons le cache si une précédente exécution de la CI l'a sauvegardé.
8
9 - name: Update Grype database
10  id: grype-db-update
11  run: |
12    mkdir -p "/cache/grype"
13    # We use Podman here since with Docker the generated cache files
14    # are root owned, causing permission issues when saving to cache.
15    update_res=$(podman run --rm -v "/cache/grype:/.cache/grype/db:Z" \
16      anchore/grype:v0.111.1 db update )
17    # Nous demandons explicitement à Grype de mettre à jour sa base de données. Ainsi,
18    # nous pouvons savoir si elle était à jour et ré-exporter le cache si nécessaire.
19    echo $update_res
20    if [ "$update_res" == "No vulnerability database update available" ]; then
21      echo "db-updated=false" >> "$GITHUB_OUTPUT"
22    else
23      echo "db-updated=true" >> "$GITHUB_OUTPUT"
24    fi
25
26 - name: Run Grype on SBOMs
27   ...
28
29 - name: Save Grype cache
30   if: "${{ always() && steps.grype-db-update.outputs.db-updated == 'true' }}"
31   uses: actions/cache/save@v5
32   with:
33     key: ${{ steps.grype-cache.outputs.cache-primary-key }}
34     path: "/cache/grype"
35     # Si la base de données a été mise à jour durant cette exécution (ligne 15), nous ré-
36     # exportons le cache.

```

Figure 21 : Code du pipeline responsable du cache de Grype

La solution retenue par les mainteneurs de SEAPATH pour la version Debian est de télécharger les images de conteneurs dans des archives avant la construction de l'image de SEAPATH, puis ensuite de les charger dans le registre local. Florent Carli, le mainteneur de SEAPATH chez RTE International, avait préalablement implémenté cette logique dans un seul des 2 scripts de lancement : `build_iso.sh`. Celui dont on a parlé dans le Chapitre 4.2 s'appelle `generate_seapath_image.sh` et ne dispose pas du code pour gérer les images de conteneurs. Il a donc fallu retravailler ce script pour permettre la gestion des images de conteneurs.

Enfin, une fois que les conteneurs sont intégrés dans l'image SEAPATH, il faut aussi les utiliser dans notre détection de vulnérabilités car ils font partie de notre pile logicielle. Nous avons donc modifié le `hook` FAI de la Figure 19 pour également scanner les images de conteneurs et générer des **SBOMs**, qu'on exporte en même temps que celui du système de fichiers. Le pipeline pour donner tous ces **SBOMs** à Grype et à VulnScout a également été modifié afin d'avoir une vue des vulnérabilités la plus complète possible.

5 VULNSCOUT

VulnScout [3] est un outil de scan et d'évaluation de vulnérabilités basé sur les [SBOMs](#). Il est fait pour être convivial pour les développeurs et simple à intégrer avec Yocto, Buildroot (un autre outil de génération d'images Linux embarquées) et avec la plupart des systèmes générant des SBOMs car il prend en charge tous les formats standards (SPDX, CycloneDX, OpenVEX). Il a été conçu pour être très facile à mettre en place avec le minimum de configuration nécessaire. Il comble ainsi le fossé entre les outils de cybersécurité, souvent complexes et difficiles à mettre en place, et le développement logiciel au quotidien, notamment dans les projets comportant des dizaines voire des centaines de dépendances open source.

Concrètement, VulnScout est une application Web hébergée localement : son backend est écrit en Python avec le framework Flask [45], tandis que le frontend est en TypeScript avec le framework React [46]. Le développement de VulnScout a commencé en mai 2024 dans le cadre du stage de fin d'études de Louis Maillard, un étudiant de l'INSA Centre Val de Loire. Il a ensuite été annoncé officiellement à Embedded World 2025 [47]. Il est depuis disponible en open source mais toujours en développement actif. Une équipe dédiée de Savoir-faire Linux Montréal travaille dessus.

Savoir-faire Linux développe également un projet connexe, *meta-vulnscout* [36]. Comme expliqué dans le Chapitre 3.1.2, il s'agit d'un *layer* Yocto contenant des classes améliorant les résultats de la classe *cve-check* de Yocto. Il contient également une classe permettant de directement lancer VulnScout depuis l'environnement de Yocto avec les [SBOMs](#) et vulnérabilités déjà importées, sans avoir besoin de manuellement déployer VulnScout à côté et de gérer l'import des données.

5.1 Analyse des vulnérabilités de SEAPATH

Il avait été décidé d'utiliser VulnScout pour étudier les vulnérabilités trouvées dans SEAPATH avant même le début du stage, afin de promouvoir l'outil auprès de la communauté Yocto mais aussi car c'est l'outil le plus adapté pour un projet basé sur Yocto. C'est donc pourquoi on a utilisé VulnScout lorsqu'il a fallu comparer les résultats des outils de détection au Chapitre 3.2.2, ou bien lorsqu'on a étudié les vulnérabilités détectées dans SEAPATH au Chapitre 3.4 et Chapitre 4.3.

Ces tâches ont principalement été menées sur la page «Vulnérabilités» de l'interface graphique (voir Figure 23a) : en filtrant uniquement sur les vulnérabilités en attente et en les triant par ordre de sévérité, il est aisé de voir les détails des vulnérabilités critiques pour ensuite chercher si elles étaient applicables ou non (voir Figure 23b). Cela a été d'une grande aide : chercher directement dans les fichiers SPDX et OpenVEX était très complexe dû à leur grande verbosité.

VulnScout étant encore en bêta, il était normal d'y trouver quelques bugs ou fonctionnalités manquantes au fil de notre utilisation. On a donc remonté ceux-ci sur le dépôt de VulnScout. Quand ceux-ci étaient bloquants pour le stage, on les a directement réglés et proposé les contributions via des [PRs](#).

En effet, notre utilisation de VulnScout est légèrement différente de celle faite par les autres équipes de Savoir-faire Linux : là où celles-ci l'utilisent directement via *meta-vulnscout*, nous paramétrons et lançons l'outil en utilisant la ligne de commande. Ainsi, nous avons pu corriger des soucis en rapport avec cette différence : par exemple, l'import des fichiers SPDX générés par SEAPATH rencontrait des problèmes de permissions. De plus, comme indiqué au Chapitre 4.3, nous ingérons le résultat d'un scan Grype directement dans VulnScout sans passer par l'outil de scan intégré, on a donc rajouté une nouvelle commande `--add-grype` à cet effet.

VulnScout SBOM Vulnerabilities Scans Review Export Settings seapath All variants						
Search Search by ID, packages, description, ... Columns Source Severity Status Published Date More Reset Filters						
	ID	Severity	EPSS Score	SBOM Affected	Variants	Actions
	CVE-2026-33845	critical	0.07%	gnutls@3.8.12 (unknown supplier)	guest_efi host_standalone_efi	Edit
	CVE-2026-5450	critical	0.07%	glibc@2.43 (unknown supplier)	guest_efi host_standalone_efi	Edit
	CVE-2026-6100	critical	0.16%	python3@3.14.4 (unknown supplier)	guest_efi host_standalone_efi	Edit
Total: 50						
1-50 / 99 - Results per page: 50 First Previous 1 2 Next Last						

(a) Page listant les vulnérabilités

CVE-2026-32746

Severity: critical

EPSS Score: 5.30%

Found by: SPDX3

Status: Not affected

Affects: inetutils@2.7 (unknown supplier)

Aliases:

Related vulnerabilities:

CVSS

CVSS 3.1

9.8

cve@mitre.org

Description

telnetd in GNU inetutils through 2.7 allows an out-of-bounds write in the LINEMODE SLC (Set Local Characters) suboption handler because add_slc does not check whether the buffer is full.

Links

https://www.cve.org/CVERecord?id=CVE-2026-32746

Assessments

2 juin 2026 à 14:55 UTC+2

inetutils@2.7 (unknown supplier)

guest_efi host_standalone_efi

Not affected - vulnerable_code_not_present

telnetd not included in SEAPATH

(b) Page montrant les détails d'une vulnérabilité ainsi que ses évaluations

Figure 23 : Extraits de l'interface graphique de VulnScout

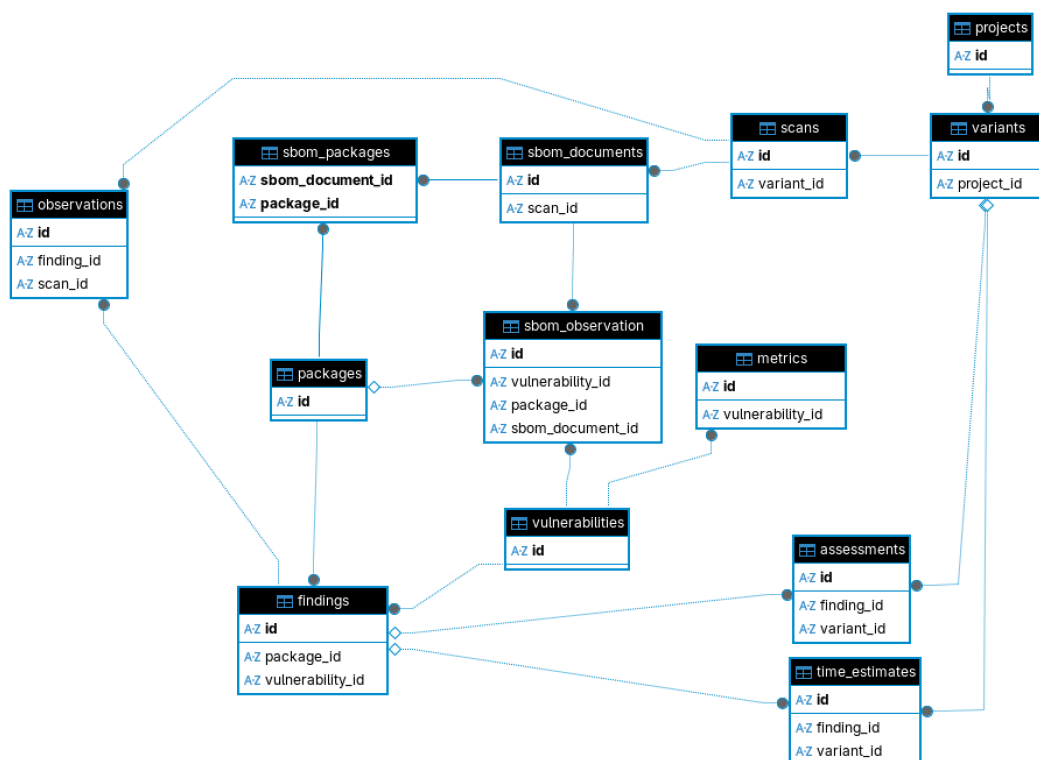


Figure 24 : Modèle entité-association de la base de données de VulnScout

Un autre soucis rencontré est la fuite de données entre projets : lorsqu'une instance VulnScout contient à la fois des projets basés sur Yocto et d'autres non, les informations provenant des SBOMs Yocto (par exemple pour indiquer qu'une vulnérabilité n'est pas présente car un fichier n'est pas compilé) sont visibles sur les pages de vulnérabilités des autres projets, alors que l'information ne les concerne pas. La correction de ce bug a nécessité de modifier le schéma de la base de données interne de VulnScout afin de stocker l'information fournie par Yocto dans une table contenant exactement les bonnes relations (la table `sbom_observation` au centre de la Figure 24). Grâce à ce changement, les informations provenant des SBOMs Yocto apparaissent sur la page des vulnérabilités seulement lorsqu'elles concernent le projet sélectionné (voir Figure 25). Ce changement a également permis d'intégrer et d'afficher les commentaires remontés par Gripe (par exemple lorsqu'une vulnérabilité est indiquée comme ignorée, voir le Chapitre 4.3).

CVE-2020-11935

Severity: medium

EPSS Score: 0.04%

Found by: Yocto

Status: Not affected

Affects: linux-mainline-rt@6.12.66-rt15 (unknown supplier)

Aliases:

Related vulnerabilities:

CVSS

CVSS 3.1

5.5

unknown

Description

It was discovered that aufs improperly managed inode reference counts in the `vfs_sub_dentry_open()` method. A local attacker could use this vulnerability to cause a denial of service attack.

Yocto Description (linux-mainline-rt)

Issue only affects aufs, which is not in SEAPATH

Figure 25 : Page montrant une information remontée par Yocto pour le projet actuel

5.2 Intégration Continue dans SEAPATH

Comme évoqué au Chapitre 3.5.2, on utilise VulnScout dans la **CI** pour générer des rapports et pour émettre une erreur lorsque des vulnérabilités dépassent les seuils de sévérité. En effet, en plus de son interface graphique facile à utiliser, VulnScout propose un outil en ligne de commande pour interagir avec, que nous utilisons dans le pipeline de détection de vulnérabilités. Une version simplifiée des commandes que nous utilisons est montrée en Figure 26.

bash

```
1 for variant in "host_standalone_efi" "guest_efi" ...; do
2   vulnscout \
3     --name "seapath" \
4     --variant "$variant" \
5     --add-spxd "$variant/sbom-cve-checked.spxd.json"
6     # On utilise les fichiers précédemment exportés par sbom-cve-check après ingestion
    des SBOMs de SEAPATH.
7 done
8 vulnscout \
9   --report "summary.adoc" \
10  --report "vulnerabilities.csv"
11
12 vulnscout \
13   --match-condition \
14   "not ignored and not fixed and (cvss >= 9.0 or (cvss >= 7.0 and epss >= 50%))" \
15   --report "failed-vulnerabilities.txt"
```

Figure 26 : Script simplifié de l'exécution de VulnScout dans la **CI**

Comme le seuil à partir duquel une vulnérabilité est considérée comme critique utilise la valeur de l'**EPSS**, VulnScout doit récupérer ces valeurs depuis la base de données en ligne, et ce pour toutes les vulnérabilités. Cette opération peut prendre plusieurs minutes selon la quantité de vulnérabilités détectées, il est donc préférable de ne pas la faire à chaque exécution de la **CI**. Pour éviter cela, on a fait en sorte de garder la base de données de VulnScout en cache pour la réutiliser entre les exécutions.

Cependant, cette base de données contient les informations sur les **SBOMs** chargés et les vulnérabilités déjà détectées aux exécutions précédentes. Conserver ces informations alors que les **SBOMs** peuvent changer peut mener à des résultats incohérents : il a donc fallu trouver un moyen de conserver uniquement les données statiques (**EPSS** et autres données sur les vulnérabilités), mais pas le reste (paquets, liste des vulnérabilités détectées, etc.)

Puisqu'il n'existait pas une telle fonctionnalité dans VulnScout, on a programmé de nouvelles commandes qui permettent de supprimer les **SBOMs** précédemment importés dans VulnScout, ainsi que les données qui y sont associées (appelées *scans*) : ainsi, la commande `vulnscout --delete-scan <scan_id>` permet de supprimer un scan à partir de son identifiant. Puisque celui-ci n'est pas nécessairement connu, il a également fallu créer la commande `vulnscout list-scans [--json]`, qui permet de lister les scans précédents et ainsi de récupérer leurs identifiants.

En **CI**, on va chaîner la commande `list-scans` à la commande `delete-scan`. Pour faire cela efficacement, on a pris soin de donner 2 modes d'affichage à la commande `list-scans`, le premier destiné à être lisible pour les humains (Figure 27a) et le deuxième fait pour être facilement interprété par les machines : lorsqu'on ajoute l'option `--json`, VulnScout affiche les scans au format JSON qu'il est aisé de manipuler programmatiquement (Figure 27b).

```
$ vulnscout list-scans
67bd6ccb-fd95-4aab-83cf-f929d1d7f207 (empty description)
at 2026-06-02 12:54:52.257306, project seapath, variant
guest_efi, 1 SBOMs, 18083 observations
a76b3f2b-488e-49cc-9808-e453fe22f630 (empty description)
at 2026-06-02 12:57:14.048960, project seapath, variant
host_standalone_efi, 1 SBOMs, 19499 observations
```

(a) Sans --json

```
$ vulnscout list-scans --json
[
  {
    "id": "67bd6ccb-fd95-4aab-83cf-f929d1d7f207",
    "description": "empty description",
    "timestamp": "2026-06-02T12:54:52.257306+00:00",
    "variant": {
      "id": "ab47745c-6a39-4eeb-b3a7-22d325fc6162",
      "name": "guest_efi",
      "project": {
        "id": "03b499ea-dd2c-40da-af36-634abae90f0d",
        "name": "seapath"
      }
    }
  },
  ...
]
```

(b) Avec --json

Figure 27 : La commande `vulnscout list-scans [--json]`

Au final, grâce à ces ajouts, notre **CI** peut réutiliser la base de données de VulnScout tant qu'elle exécute ces commandes au début, à la manière décrite dans le script de la Figure 28.

```
bash
1 old_scans=$(vulnscout --list-scans --json | jq ".[] | .id" -r)
  # jq est un processeur de texte JSON. On s'en sert ici pour récupérer la liste des champs
  id pour chaque scan.
2
3 for old_scan_id in $old_scans; do
4   vulnscout --delete-scan "$old_scan_id"
5 done
```

Figure 28 : Script permettant de supprimer tous les scans de la base de données de VulnScout

5.3 Amélioration de la qualité du code

Comme introduit au début du Chapitre 5, VulnScout est à la base un projet développé durant un stage, puis repris par différents collaborateurs de Savoir-faire Linux. Malgré sa relative jeunesse, une certaine quantité de dette technique s'est accumulée, surtout due à l'utilisation de conventions de code datées et de frameworks obsolètes. De plus, l'équipe actuelle de Savoir-faire Linux Montréal travaillant sur VulnScout doit avancer à une certaine allure pour rajouter des fonctionnalités et dispose de peu de temps pour retoucher l'ancien code.

Le principal souci identifié concerne l'absence quasi complète de typage dans le backend. En effet, Python n'est pas un langage typé explicitement : on n'a pas besoin de spécifier le type de ses variables, le type des paramètres des fonctions ou bien le type de retour d'une méthode. Cela en fait un langage facile à prendre en main et rapide à écrire, mais cela a l'inconvénient de rendre l'analyse statique de code très complexe : là où le compilateur d'un langage typé statiquement comme le Java peut facilement détecter l'accès à une variable inconnue ou le passage d'un nombre dans une fonction s'attendant à une chaîne de caractère, les outils dédiés pour Python ont beaucoup de mal à faire ces mêmes analyses. Pour pallier ces problèmes, depuis la version 3.0 de Python (2008), les développeurs peuvent ajouter des annotations de type. Ce n'est pourtant toujours pas fait partout, et c'est le cas dans VulnScout.

Tout le code créé pour VulnScout dans le cadre de ce stage a donc été écrit de la manière la plus propre possible, avec un typage explicite et en utilisant les fonctionnalités récentes des

langages et frameworks utilisés. De plus, on a pris soin de revoir les [PRs](#) des autres contributeurs afin de s'assurer que code nouvellement introduit soit également de bonne qualité. La politique du projet VulnScout étant d'avoir obligatoirement 2 revues par des personnes différentes pour chaque [PR](#), on a eu l'occasion de revoir la plupart d'entre elles (la taille de l'équipe étant assez réduite, oscillant entre 3 et 4 personnes en nous comptant).

Afin de mesurer l'impact que nous avons eu sur la qualité du code, nous avons utilisé les outils *mypy* et *Pyright* pour calculer des estimateurs sur le typage du code : pourcentage d'imprécision dans les fichiers, nombre de fichiers avec des erreurs de typage explicite, nombre de symboles (variables, fonctions, etc.) avec des types connus, etc. On a effectué ces mesures sur deux versions de VulnScout : la version 0.10 qui était la dernière lorsque le stage a commencé, puis la version 0.15 qui est la version courante lors de l'écriture de ce rapport.

Tableau 4 : Évolution des indicateurs de qualité du typage du backend au cours du stage

Version	Lignes de code	mypy		Pyright	
		Imprécision	Fichiers erronés	Exhaustivité du typage	Symboles avec types connus
0.10	6214	37,51 %	8 / 43	32,9 %	156 / 430
0.15	18630	34,97%	7 / 99	34,5 %	316 / 861

Une première chose à remarquer dans le Tableau 4 est que la base de code du backend a triplé, en passant d'environ 6 000 à 18 000 lignes, tandis que le nombre de fichier a plus que doublé (de 43 à 99). Malgré cela, il y a moins de fichiers avec des erreurs de typage, et la précision et l'exhaustivité du typage ont augmenté de quelques points de pourcentage.

Ces évolutions ne semblent pas très grandes, mais mises en relation avec le triplement du nombre de lignes de codes, cela montre l'effort mis dans la revue des [PRs](#) pour que le code nouvellement intégré soit le plus correctement typé. De plus, il y a eu très peu de temps destiné à l'amélioration pure du code existant, le nouveau code dépend donc parfois de l'ancien code non typé et est donc imprécis par effet boule de neige.

Au final, grâce à ces améliorations, il est plus aisé et sûr d'écrire du nouveau code dans VulnScout car on peut savoir le type des variables et fonctions qu'on appelle et donc éviter des erreurs simples.

5.4 Comparaison avec Dependency-Track

Dependency-Track [48] est une plate-forme d'analyse de composants logiciels permettant d'identifier les risques dans une chaîne d'approvisionnement logicielle. En pratique, elle se base sur les [SBOMs](#) pour détecter les vulnérabilités présentes dans un système. C'est donc un clair concurrent de VulnScout.

À plusieurs reprises, sur des salons où ils présentaient VulnScout, des membres de Savoir-faire Linux ont été questionnés sur la différence avec Dependency-Track et l'intérêt de VulnScout en comparaison. Une partie du stage a donc été consacrée à l'étude et à la comparaison en profondeur des deux outils, du déploiement initial à l'expérience d'évaluation de vulnérabilités, en passant par l'intégration avec les projets basés sur Yocto.

On a ensuite écrit un rapport détaillé sur cette comparaison. On a pris soin de consigner les points faibles et forts des deux outils, afin de pouvoir en tirer une liste d'améliorations possibles pour VulnScout. De plus, on a souligné les profondes différences de philosophies, qui font que ces outils ne sont pas en réelle concurrence car ils n'ont pas les mêmes utilisateurs cibles. Le rapport

a été partagé à l'équipe du projet VulnScout de Montréal. Il est prévu d'ensuite le publier en article de blogue sur le site Web de Savoir-faire Linux. Le rapport est consultable dans l'Annexe 3.

6 CONCLUSION

Ce projet de fin d'études a permis d'entamer le processus de gestion des vulnérabilités dans le projet SEAPATH, en mettant en place une génération de [SBOMs](#) et une détection automatisée dans la [CI](#). Ces travaux ouvriront la voie à de futures discussions sur la gestion de la cybersécurité dans le projet SEAPATH, dans l'optique d'être cohérent avec les futures législations du domaine de l'informatique comme le CRA [26].

Ces travaux ont également permis de rendre publique la [CI](#) de SEAPATH qui était encore interne chez Savoir-faire Linux. Ils rendent donc la chaîne d'intégration continue plus transparente, en plus d'offrir la possibilité dans le futur de publier les nouvelles versions de SEAPATH complètement automatiquement de manière traçable.

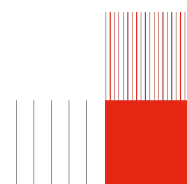
Cette expérience sur SEAPATH m'a permis de découvrir le développement dans le cadre d'un projet [open source](#) mené par une organisation, où il faut composer avec différents acteurs et où les décisions ne se prennent pas aussi rapidement que pour un projet classique. De plus, j'ai pu contribuer à un projet open source de grande envergure, Yocto, via leur liste de diffusion, ce qui était assez impressionnant et m'a beaucoup appris sur la manière de fonctionner des communautés de l'open source.

En parallèle, j'ai pu participer au cycle de développement d'un projet logiciel open source, VulnScout, d'une position privilégiée en étant à la fois utilisateur et contributeur du projet. Cela m'a permis de développer de précieuses compétences en collaboration sur un projet agile via les réunions hebdomadaires et les revues de code.

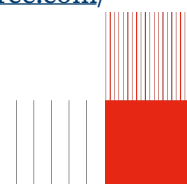
Enfin, j'ai eu la chance d'évoluer dans un environnement d'une très grande compétence technique que j'ai pu en partie acquérir grâce aux réunions quotidiennes, aux revues de code des collègues et aux discussions autour de la table du déjeuner.

BIBLIOGRAPHIE

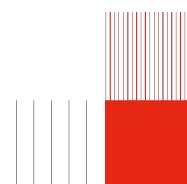
- [1] SDES, « Chiffres clés des énergies renouvelables », 2025. Consulté le: 13 mai 2026. [En ligne]. Disponible sur: <https://www.statistiques.developpement-durable.gouv.fr/edition-numerique/chiffres-cles-energies-renouvelables/fr/>
- [2] LF Energy, « SEAPATH ». Consulté le: 21 avril 2026. [En ligne]. Disponible sur: <https://lfenergy.org/projects/seapath/>
- [3] « VulnScout: The Easy-to-Use Tool for Securing Embedded Systems and Staying Compliant », Savoir-faire Linux. Consulté le: 23 avril 2026. [En ligne]. Disponible sur: <https://blog.savoirfairelinux.com/en-ca/2025/vulnscout-the-easy-to-use-tool-for-securing-embedded-systems-and-staying-compliant/>
- [4] The Debian Project, « Debian Reference ». Consulté le: 21 avril 2026. [En ligne]. Disponible sur: <https://www.debian.org/doc/manuals/debian-reference/>
- [5] M. Garrels, *Introduction to Linux (Third Edition)*, 3^e éd. Fultus Corporation, 2010. [En ligne]. Disponible sur: <https://books.google.fr/books?id=z3MoAH32jzIC>
- [6] « Jami ». Consulté le: 18 mai 2026. [En ligne]. Disponible sur: <https://jami.net/>
- [7] D. Spinellis, « Git », *IEEE Software*, vol. 29, n° 3, p. 100-101, 2012, doi: [10.1109/MS.2012.61](https://doi.org/10.1109/MS.2012.61).
- [8] M. Mukadam, C. Bird, et P. C. Rigby, « Gerrit software code review data from Android », in *2013 10th Working Conference on Mining Software Repositories (MSR)*, 2013, p. 45-48. doi: [10.1109/MSR.2013.6624002](https://doi.org/10.1109/MSR.2013.6624002).
- [9] « GitHub ». Consulté le: 18 mai 2026. [En ligne]. Disponible sur: <https://github.com/>
- [10] V. Armenise, « Continuous Delivery with Jenkins: Jenkins Solutions to Implement Continuous Delivery », in *2015 IEEE/ACM 3rd International Workshop on Release Engineering*, 2015, p. 24-27. doi: [10.1109/RELENG.2015.19](https://doi.org/10.1109/RELENG.2015.19).
- [11] C. Chandrasekara et P. Herath, *Hands-on GitHub Actions: Implement CI/CD with GitHub Action Workflows for Your Applications*. Berkeley, CA: Apress, 2021. doi: [10.1007/978-1-4842-6464-5](https://doi.org/10.1007/978-1-4842-6464-5).
- [12] Savoir-faire Linux, *cqfd*. Consulté le: 18 mai 2026. [En ligne]. Disponible sur: <https://github.com/savoirfairelinux/cqfd>
- [13] Savoir-faire Linux, *Cukinia*. Consulté le: 18 mai 2026. [En ligne]. Disponible sur: <https://github.com/savoirfairelinux/cukinia>
- [14] Shigeru23, « Electrical substation model (side-view) ». Consulté le: 18 mai 2026. [En ligne]. Disponible sur: [https://commons.wikimedia.org/wiki/File:Electrical_substation_model_\(side-view\).PNG](https://commons.wikimedia.org/wiki/File:Electrical_substation_model_(side-view).PNG)
- [15] J. McDonald, « Substation Automation Basics - The Next Generation », *Electric Energy T&D Magazine*, p. 30-36, 2007, Consulté le: 22 mai 2026. [En ligne]. Disponible sur: <https://electricenergyonline.com/energy/magazine/321/article/Substation-Automation-Basics-The-Next-Generation.htm>
- [16] N. Kabbara *et al.*, « Towards Software-Defined Protection, Automation, and Control in Power Systems: Concepts, State of the Art, and Future Challenges », *Energies*, vol. 15, n° 24, 2022, doi: [10.3390/en15249362](https://doi.org/10.3390/en15249362).
- [17] A. Kivity, Y. Kamay, D. Laor, U. Lublin, et A. Liguori, « kvm: the Linux virtual machine monitor », in *Proceedings of the Linux symposium*, 2007, p. 225-230. Consulté le: 18 mai 2026. [En ligne]. Disponible sur: <https://www.kernel.org/doc/mirror/ols2007v1.pdf#page=225>
- [18] « QEMU ». Consulté le: 18 mai 2026. [En ligne]. Disponible sur: <https://www.qemu.org/>
- [19] M. Bolte, M. Sievers, G. Birkenheuer, O. Niehörster, et A. Brinkmann, « Non-intrusive virtualization management using libvirt », in *2010 Design, Automation & Test in Europe Conference & Exhibition (DATE 2010)*, 2010, p. 574-579. doi: [10.1109/DATE.2010.5457142](https://doi.org/10.1109/DATE.2010.5457142).



- [20] B. Pfaff *et al.*, « The Design and Implementation of Open vSwitch », in *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI 15)*, Oakland, CA: USENIX Association, mai 2015, p. 117-130. [En ligne]. Disponible sur: <https://www.usenix.org/conference/nsdi15/technical-sessions/presentation/pfaff>
- [21] ClusterLabs, « Pacemaker ». Consulté le: 18 mai 2026. [En ligne]. Disponible sur: <https://clusterlabs.org/projects/pacemaker/>
- [22] Corosync. Consulté le: 18 mai 2026. [En ligne]. Disponible sur: <https://github.com/corosync/corosync>
- [23] « Ceph ». Consulté le: 18 mai 2026. [En ligne]. Disponible sur: <https://ceph.com/en/>
- [24] The Yocto Project®, « Yocto Project Documentation ». Consulté le: 21 avril 2026. [En ligne]. Disponible sur: <https://docs.yoctoproject.org/>
- [25] L. Hochstein et R. Moser, *Ansible: Up and Running: Automating Configuration Management and Deployment the Easy Way*. O'Reilly Media, 2017. [En ligne]. Disponible sur: <https://books.google.fr/books?id=h5YtDwAAQBAJ>
- [26] C. Schmittner, O. Veledar, T. Faschang, G. Macher, et E. Brenner, « Fostering Cyber Resilience in Europe: An In-Depth Exploration of the Cyber Resilience Act », in *Systems, Software and Services Process Improvement*, M. Yilmaz, P. Clarke, A. Riel, R. Messnarz, C. Greiner, et T. Peisl, Éd., Cham: Springer Nature Switzerland, 2024, p. 390-404.
- [27] M. Mirakhorli *et al.*, « A Landscape Study of Open Source and Proprietary Tools for Software Bill of Materials (SBOM) ». [En ligne]. Disponible sur: <https://arxiv.org/abs/2402.11151>
- [28] Trex Team, *Grype in Production*. NobleTrex Press, 2026. Consulté le: 21 avril 2026. [En ligne]. Disponible sur: https://books.google.com/books/about/Grype_in_Production.html?hl=fr&id=LJ_KEQAAQBAJ
- [29] bootlin, *sbom-cve-check*. Consulté le: 21 avril 2026. [En ligne]. Disponible sur: <https://github.com/bootlin/sbom-cve-check/>
- [30] H. Booth, « National Vulnerability Database ». Consulté le: 11 mars 2026. [En ligne]. Disponible sur: <https://nvd.nist.gov/>
- [31] CVEProject, *cvelistV5*. Consulté le: 11 mars 2026. [En ligne]. Disponible sur: <https://github.com/CVEProject/cvelistV5>
- [32] Linux, *vulns*. Consulté le: 11 mars 2026. [En ligne]. Disponible sur: <https://git.kernel.org/pub/scm/linux/security/vulns.git/>
- [33] Debian, « Security Bug Tracker ». Consulté le: 26 mai 2026. [En ligne]. Disponible sur: <https://security-tracker.debian.org/tracker/>
- [34] RedHat, « Security Updates ». Consulté le: 26 mai 2026. [En ligne]. Disponible sur: <https://access.redhat.com/security/security-updates/>
- [35] « Classes - cve-check ». Consulté le: 11 mars 2026. [En ligne]. Disponible sur: <https://docs.yoctoproject.org/scarthgap/ref-manual/classes.html#ref-classes-cve-check>
- [36] *meta-vulnscout*. Consulté le: 11 mars 2026. [En ligne]. Disponible sur: <https://github.com/savoirfairelinux/meta-vulnscout/tree/scarthgap>
- [37] GitHub, « Reuse workflows - GitHub Actions Docs ». Consulté le: 2 juin 2026. [En ligne]. Disponible sur: <https://docs.github.com/en/actions/how-tos/reuse-automations/reuse-workflows>
- [38] SEAPATH, *yocto-bsp*. Consulté le: 28 mai 2026. [En ligne]. Disponible sur: <https://github.com/seapath/yocto-bsp>
- [39] SEAPATH, *meta-seapath*. Consulté le: 28 mai 2026. [En ligne]. Disponible sur: <https://github.com/seapath/meta-seapath>
- [40] Google, *git-repo*. Consulté le: 28 mai 2026. [En ligne]. Disponible sur: <https://gerrit.googlesource.com/git-repo/>

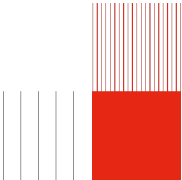
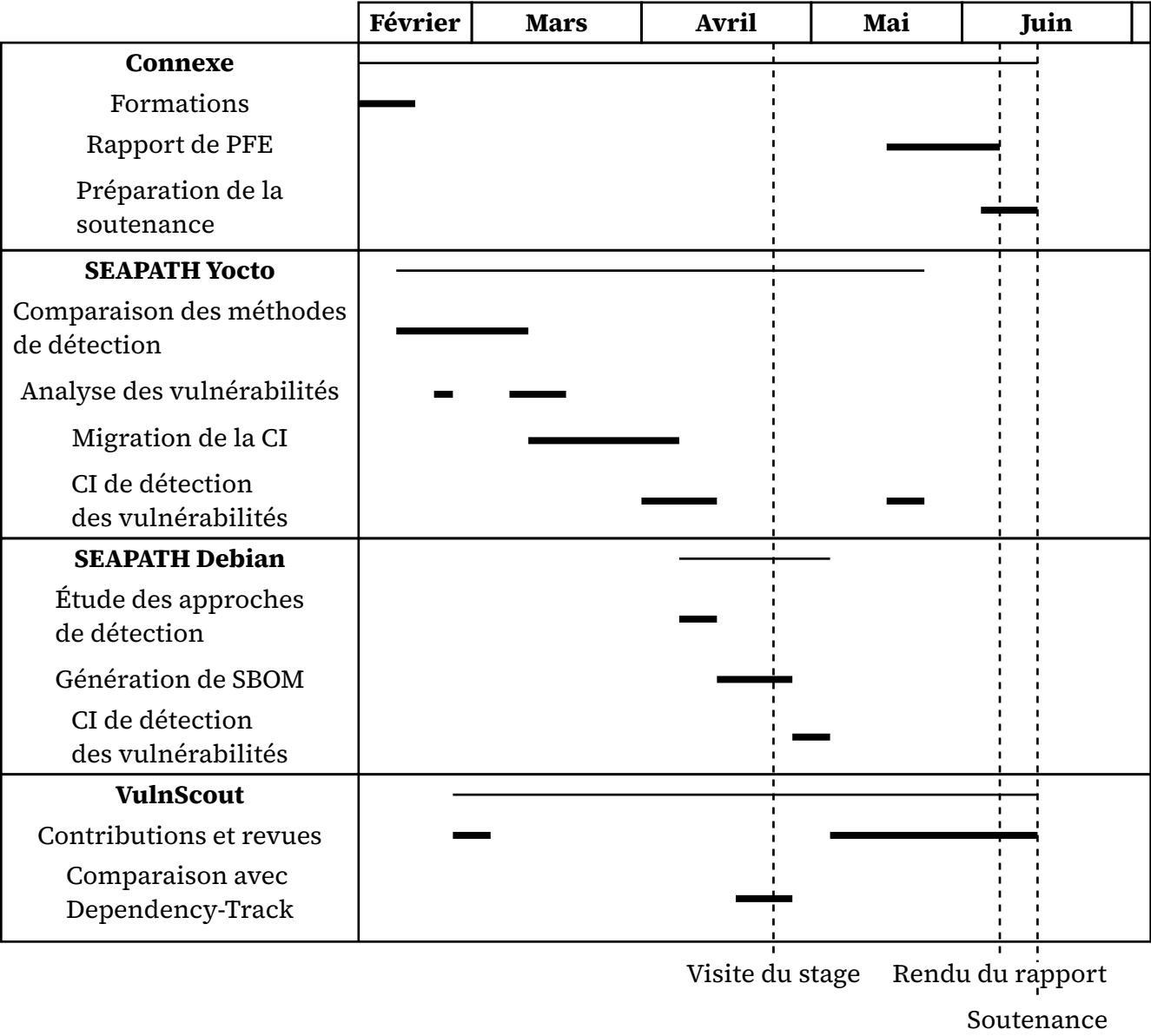


- [41] SEAPATH, *repo-manifest*. Consulté le: 28 mai 2026. [En ligne]. Disponible sur: <https://github.com/seapath/repo-manifest>
- [42] Thomas Lange, « FAI - Fully Automatic Installation ». Consulté le: 4 juin 2026. [En ligne]. Disponible sur: <https://fai-project.org/>
- [43] Anchore, *Syft*. Consulté le: 23 avril 2026. [En ligne]. Disponible sur: <https://github.com/anchore/syft>
- [44] *Podman*. Consulté le: 5 juin 2026. [En ligne]. Disponible sur: <https://github.com/podman-container-tools/podman>
- [45] Pallets, « Flask Documentation ». Consulté le: 3 juin 2026. [En ligne]. Disponible sur: <https://flask.palletsprojects.com/en/stable/>
- [46] Facebook, *React*. Consulté le: 3 juin 2026. [En ligne]. Disponible sur: <https://github.com/facebook/react>
- [47] Savoir-faire Linux, *Cybersécurité : Savoir-faire Linux a annoncé à Embedded World 2025 la sortie en Open Source de VulnScout.io*. 2025. Consulté le: 2 juin 2026. [En ligne]. Disponible sur: https://www.images-et-reseaux.com/voy_content/uploads/2025/04/sfl-vulnscout_io-pr-20250311_fr-2.pdf
- [48] « Dependency-Track ». Consulté le: 22 avril 2026. [En ligne]. Disponible sur: <https://dependencytrack.org/>



ANNEXES

Annexe 1 - Planning



Annexe 2 - Rapport de comparaison des solutions de détection de vulnérabilité

Savoir-faire Linux

Youenn Le Jeune

CVE detection for Yocto

Vulnerabilities assessment is a crucial aspect of developing and maintaining a software project. If you are using Yocto, there are multiple possible approaches to detect and filter CVEs in a Yocto-built image. This report will compare some of them and present data, analysis and recommendations.

1. Analysis details

- All of the detection runs have been made on the same version of SEAPATH [1] on the 2026-03-10.
- SEAPATH is not based on a linux-yocto kernel but on a custom recipe, using the mainline-linux-rt kernel. Some details are provided in [Section 3](#).
- We are using Yocto Scarthgap, the current LTS version at the time of writing.
- The kernel version is 6.12.66, a very recent revision at the moment the analysis was done.
- Runs have been made with only a few minutes intervals between them, meaning the probability of new CVEs being discovered in-between is slim.

2. Tested tools and scenarios

2.1. “Vanilla” Yocto cve-check

This is the most basic scenario: we inherit the `cve-check` class from the Poky layer [2], and we include the `cve-extra-exclusions.inc` file to filter some non-applicable CVEs.

2.2. “Light” meta-vulnscout

Savoir-faire Linux’s [3] VulnScout [4] team has developed a Yocto layer that contains a lot of useful classes to improve the vanilla `cve-check` results: `meta-vulnscout` [5].

In this scenario, we will only use 2 of those classes: `kernel_generate_cve_exclusions` and `improve_kernel_cve_report`, which are based on similarly-named scripts in the OpenEmbedded-Core layer (more details on `meta-vulnscout` GitHub page).

2.3. “Full” meta-vulnscout

This scenario is based on the previous one, with the addition of the `kernel_filter_nonbuilt_cves` class crafted by the VulnScout team.

2.4. “External” sbom-cve-check

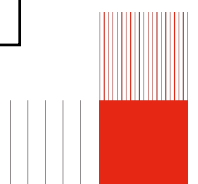
The Bootlin [6] team has created `sbom-cve-check` [7], an util to detect CVEs from SBOM files. It can work with files generated by a Yocto build, without having to run during the build process. We are using the `create-spx-2.2` and `vex` classes to generate the input files.

3. Details on the Linux Kernel recipe

The `linux-yocto` kernel recipe references some extra exclusion files¹ that filter a lot of kernel CVEs by default. Since SEAPATH is using a custom kernel recipe, we had to integrate those utils in our recipe, otherwise we would have got a lot of false positives.

One of those 2 exclusion files, `cve-exclusion_6.18.inc`, is generated by a script using the Linux Kernel “vulns” repository [8] as a source. We can ignore it since it’s already a data source of our tools, as we are going to see later. The problem lies in the other file, `cve-exclusion.inc`: it contains CVE statuses for CVEs that are *not* completely documented in the vulns repository. Therefore, we

¹The `cve-exclusion` files from <https://git.openembedded.org/openembedded-core/tree/meta/recipes-kernel/linux?id=699dbbdf3ab2693bae8a7e0425e2519250fdfec4&h=scarthgap>



integrated this exclusion file in SEAPATH, after splitting it to suit our kernel version needs. The commit for this work is available on GitHub: [SEAPATH/meta-seapath@bbc1fc1](https://github.com/SEAPATH/meta-seapath@bbc1fc1).

One last thing we had to do is to make the built kernel sources included in the generated SPDX file, so `sbom-cve-check` can filter non-built kernel CVEs based on that. In `linux-yocto` it's simply a matter of enabling the `SPDX_INCLUDE_COMPILED_SOURCES` variable, but it relies on some custom kernel configuration that is present by default in `linux-yocto` and that we had to include in our recipe. The commit that enabled this is also available on GitHub: [SEAPATH/meta-seapath@abd2818](https://github.com/SEAPATH/meta-seapath@abd2818).

4. Results

4.1. Total amounts

The first interesting thing to notice in [Figure 1](#) and [Table 1](#) is the total volume of CVEs detected by the tools: in comparison with the vanilla `cve-check` tool, both the `meta-vulnscout` and the `sbom-cve-check` utils detected roughly 2000 more CVEs. This is easily explainable since `cve-check`'s only database is the NVD [9] which is not complete, especially concerning the Linux kernel -related CVEs. `meta-vulnscout` classes uses the same data as `cve-check` *plus* the `cvelistV5` database [10] and the official kernel “vulns” repository [8]. On the other hand, `sbom-cve-check` only uses `cvelistV5` and the NVD database (but `cvelistv5` should include all CVEs from the kernel vulns²). A summary is available in [Table 2](#).

	fixed	not_affected	under_investigation	total
vanilla	14 957	95	131	15 183
light	16 534	378	363	17 275
full	16 534	453	287	17 274
external	16 520	403	120	17 043

Table 1: Raw CVE statuses data

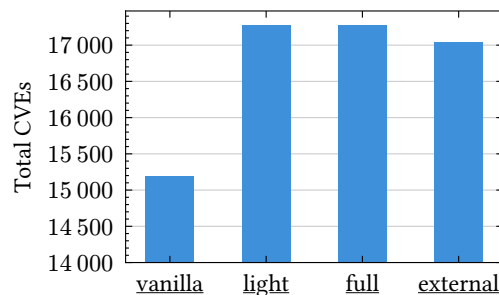


Figure 1: Total amount of detected CVEs, per detection scenario.

Beware, the *y* axis does not start at 0.

	NVD	cvelistv5	vulns	
cve-check	✓	✗	✗	Fails to link some CVEs with recipes
meta-vulnscout	✓	✓	✓	Exports everything
sbom-cve-check	✓	✓	✗	Does not export rejected CVEs

Table 2: Summary of CVE origin databases

²<https://www.cve.org/PartnerInformation/ListofPartners/partner/Linux>

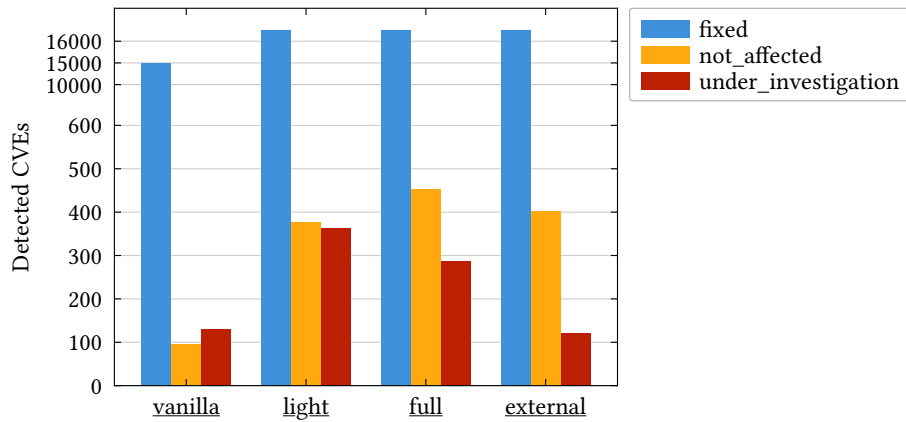


Figure 2: Statuses of the detected CVEs, per detection scenario.

Mind the y axis for the large “fixed” amounts.

Another thing to notice in the total amount of detected CVEs is that `sbom-cve-check` has detected ~230 less CVEs than `meta-vulnscout`. It is actually because the former automatically discards CVEs that have been rejected by CNAs (such as [CVE-2021-47084](#)) whereas the latter keeps them as “not affected”, thus its total amount is bigger.

4.2. Assessment size

The most interesting data to us is the amount of CVEs that are “under investigation”. It tells us how much time we should spend on our assessment. The smaller the better, as long as it does not have false negatives.

We can already say that the `vanilla cve-check` contains a lot of false negatives since it’s based on only 1 incomplete database, as we have seen in the previous section.

Concerning the `meta-vulnscout` scenarios, it is pretty clear that adding the `kernel_filter_nonbuilt_cves` class decreases a lot the amount of CVEs to assess: as shown in [Table 1](#), the `full` scenario has ~20% less CVEs in the “under investigation” status than the `light` one. And this is for a recent kernel version, for ancient versions it can be way higher: we made the same comparison for a version of SEAPATH that is more than 1 year old (SEAPATH 1.0.0 from January 22th, 2025) and we get 70% less CVEs to investigate! See the raw data in [Table 3](#).

	fixed	not_affected	under_investigation	total
vanilla	12 524	81	2552	15 157
full	13 925	1670	765	16 360

Table 3: Raw CVE statuses data for SEAPATH 1.0.0

`sbom-cve-check` produces even less CVEs to assess. From our analysis, it seems to be thanks to its better filtering on non-built kernel CVEs (e.g. CVE-2025-40338 which `sbom-cve-check` correctly flags as “compiled code not present” whereas `meta-vulnscout` does not). In terms of CVEs reported, no matter their statuses, it looks like `sbom-cve-check` includes *all* the CVEs from `meta-vulnscout` with some more. You can see the differences in assessment between the `full` and `external` scenarios in details in [Section 6.2](#).

Unfortunately we could not try `sbom-cve-check` on the old SEAPATH 1.0.0 as we did with `meta-vulnscout` since it requires the `vex` class, not present in the Yocto Scarthgap revision in use in 1.0.0.

5. Conclusion

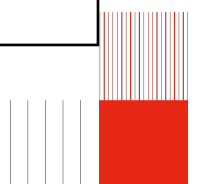
We compared the default `cve-check` tool embedded in Yocto with Savoir-faire Linux's `meta-vulnscout` [5] and Bootlin's `sbom-cve-check` [7] to see which ones detected the most vulnerabilities and how did they perform filtering non-applicable ones. Both tools detect many more CVEs than only using `cve-check` which is a really great point to have an accurate view of the vulnerabilities of a Yocto image, but Bootlin's tool really shines when it comes to filtering non-applicable or fixed ones, which reduces the time to spend doing the assessment by hand.

Furthermore, another big advantage of `sbom-cve-check` is its ability to run outside of Yocto: the SBOM is generated by Bitbake during the build, then you can run the CVE check again at different points in time without having to re-launch the build. This can be very valuable to periodically follow the evolution of vulnerabilities in a product that has been deployed, in a CI for example.

`meta-vulnscout` still holds a big plus for users that want to do their vulnerabilities assessments directly from the build environment: the meta layer can directly generate a VulnScout [4] config file and run it as a Bitbake task, whereas for `sbom-cve-check` you have to manually pass the generated files to the tool and then pass the newly exported files to your CVE assessment tool, which can be cumbersome.

Bibliography

- [1] "SEAPATH." Accessed: Mar. 11, 2026. [Online]. Available: <https://lfenergy.org/projects/seapath/>
- [2] "Classes - cve-check." Accessed: Mar. 11, 2026. [Online]. Available: <https://docs.yoctoproject.org/scarthgap/ref-manual/classes.html#ref-classes-cve-check>
- [3] Savoir-faire Linux, "Savoir-faire Linux | Leader in Open Source Technologies and Free Software." Accessed: Mar. 11, 2026. [Online]. Available: <http://savoirfairelinux.com/>
- [4] *VulnScout*. Accessed: Mar. 11, 2026. [Online]. Available: <https://github.com/savoirfairelinux/vulnscout/>
- [5] *meta-vulnscout*. Accessed: Mar. 11, 2026. [Online]. Available: <https://github.com/savoirfairelinux/meta-vulnscout/>
- [6] Bootlin, "Bootlin – Embedded Linux and kernel engineering." Accessed: Mar. 11, 2026. [Online]. Available: <https://bootlin.com/>
- [7] T. Petazzoni, "Announcing sbom-cve-check, a lightweight CVE analysis tool for your SBOM," Bootlin. Accessed: Mar. 11, 2026. [Online]. Available: <https://bootlin.com/blog/announcing-sbom-cve-check-a-lightweight-cve-analysis-tool-for-your-sbom/>
- [8] *vulns*. Accessed: Mar. 11, 2026. [Online]. Available: <https://git.kernel.org/pub/scm/linux/security/vulns.git/>
- [9] H. Booth, "National Vulnerability Database." Accessed: Mar. 11, 2026. [Online]. Available: <https://nvd.nist.gov/>
- [10] *cvelistV5*. Accessed: Mar. 11, 2026. [Online]. Available: <https://github.com/CVEProject/cvelistV5>



Annexe 3 - Rapport de comparaison entre VulnScout et Dependency-Track

Savoir-faire Linux

Youenn Le Jeune

Yocto vulnerability assessment: VulnScout or Dependency-Track?

Yocto-based projects are huge software projects that have huge dependency trees and that can have hundreds or thousands of vulnerabilities. Assessing them is crucial (and soon to be mandatory in Europe¹). There are several software solutions available for vulnerability assessment and this paper will compare 2 of them: VulnScout^[1] developed by Savoir-Faire Linux and Dependency-Track^[2] developed by the OWASP Foundation. The comparison will especially focus on differences that matter for Yocto^[3] based projects.

1. Design philosophies

Both tools have a fundamental design difference: Dependency-Track is made to be deployed on a server in a company's infrastructure and is to be used by multiple people. It can be deployed once and centralize all projects of the company in a single place. VulnScout is very different: it is made to be ran on the user's machine only for the duration of its use and to be stopped afterwards.

This design difference is obvious when you look at the out-of-box experience: to launch VulnScout, all you have to do is clone the repository, run `./vulnscout --serve` and wait a few seconds. To start Dependency-Track, you have to download a Docker Compose file, run it then wait dozens of minutes for the databases to be mirrored (when testing it to write this report, it took a whole 1h30m). Then you'll have to set-up administrative credentials, and finally you'll be able to access the application.

This difference alone can already contribute to the choice of assessment tool: for instance, when you only have to do assessments once every few months for a few small to medium projects, it might not be worth it to deploy a Dependency-Track instance. VulnScout can perfectly handle that. Another example would be an open-source project: if you want your external contributors or end users to be able to completely reproduce the build workflow, from the source fetch to the vulnerability assessment on the build artifacts, you'll have to either give access to your Dependency-Track instance (which might not be possible) or require external users to install it themselves (which is cumbersome as we saw earlier). VulnScout is better suited for this. Now, if you handle several big, closed source projects and you need to constantly keep a global view on the vulnerabilities, it might be better to deploy Dependency-Track at your company and centralize your projects there.

2. Features comparison

2.1. SBOM formats

Dependency-Track historically supported SPDX2 as an import format but it has been removed for political and technical reasons². While SPDX3 support is being considered³, as of now, Dependency-Track can only import CycloneDX SBOM and VEX files (see [Figure 1](#)). It never supported the OpenVEX format⁴. This is unfortunate since Yocto can natively output SBOM as SPDX2 and/or SPDX3 files (depending on the Yocto version) and CVE statuses as OpenVEX files. While there are some ways to circumvent this issue, they are not perfect:

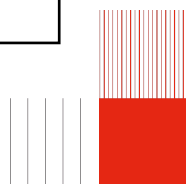
- Some tools allow to convert SPDX files to CycloneDX files. However, they usually lose quite a lot of information.

¹<https://digital-strategy.ec.europa.eu/en/policies/cyber-resilience-act>

²<https://github.com/DependencyTrack/dependency-track/discussions/1222>

³<https://github.com/DependencyTrack/dependency-track/issues/1746>

⁴<https://github.com/DependencyTrack/dependency-track/issues/4862#issuecomment-2820847602>



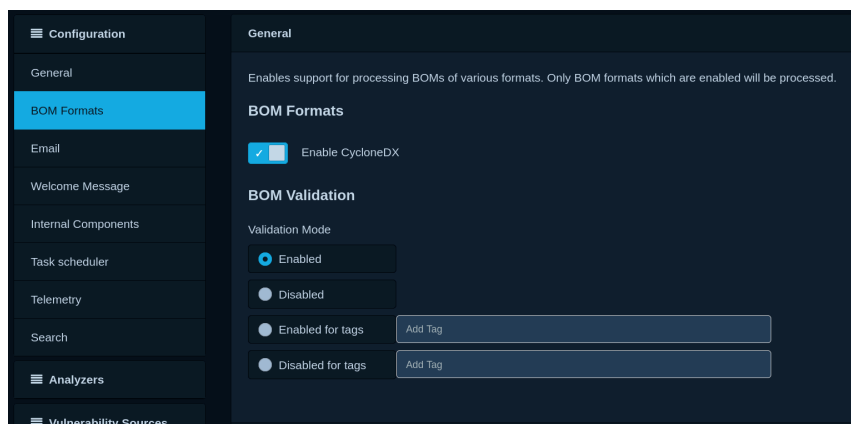


Figure 1: Dependency-Track “BOM Formats” configuration page showing only CycloneDX

- iris-GmbH maintains *meta-cyclonedx* [4] as a way to export CycloneDX files from Yocto. However, they do not contain as much data as the SPDX files natively generated by Yocto: for instance, there is no way to export the list of compiled files that would allow to filter out a lot of Linux Kernel vulnerabilities (but some work seems to be ongoing in a forked repository⁵).

On the contrary, VulnScout support a lot of SBOM formats: SPDX2 (both individual JSON files and archive files produced by Yocto Scarthgap), SPDX3, CycloneDX, OpenVEX, output of Grype scans, output of Yocto’s `cve-check` class. Basically, VulnScout can be directly wired to the output of a Yocto build without complicated manipulations.

2.2. Reports

VulnScout has an interesting feature: you can pass it Jinja templates (named “reports”) and it will *render* them using an internal set of variables spanning vulnerabilities, projects, packages and so on. It is really interesting in order to produce an artifact than can be attached to builds and given to end users so they are informed about the packaged software and the detected vulnerabilities in full transparency. Those reports can be exported in textual formats or rendered in PDF or HTML files from AsciiDoc files (see Figure 2). It looks like Dependency-Track does not have such a feature.

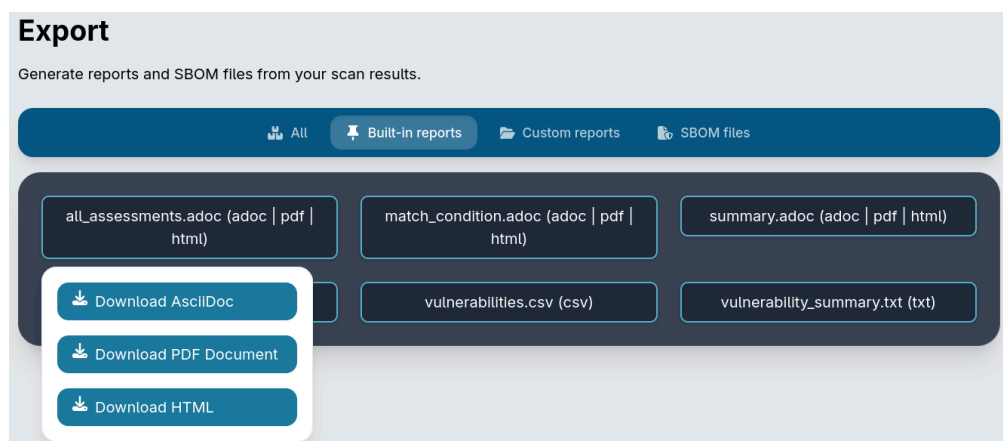


Figure 2: VulnScout “export” tab

⁵<https://github.com/iris-GmbH/meta-cyclonedx/issues/53>

2.3. Variants and Versions

VulnScout and Dependency-Track have the same feature but named differently, respectively “Variants” and “Versions”. These essentially allow users to upload different SBOMs to the same project and have different vulnerabilities assessments. This is useful for example when working on Yocto-based projects that have different versions (machine-distribution-image couples).

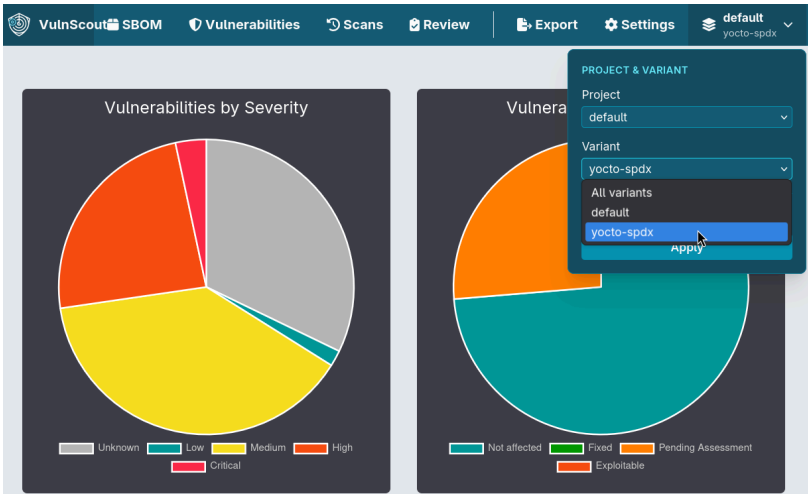
In practice however they are not treated in the same way. In Dependency-Track, having the same vulnerability in 2 different versions essentially means having 2 vulnerabilities: it is counted twice in every vulnerabilities counter. Thus, if you add a new version of your package with only a few packages more, your vulnerabilities count will basically double. The only place where the vulnerabilities are actually grouped by project is in the “grouped vulnerabilities” tab (see [Figure 3](#)) which unfortunately contains the vulnerabilities of all projects without means to filter them.

On the other hand, if VulnScout sees the same vulnerability in 2 variants of the same project, it will only count it once in the dashboard graphs. If users want to get counts for only one variant, they can select it in the variants picker (see [Figure 4a](#)). Furthermore, in the “vulnerabilities” page, the variants it applies too appear in a column (see [Figure 4b](#)). Unlike in Dependency-Track, it directly shows the variant names instead of only a number.



Vulnerability	Severity	Published	CWE	CVSSv2	Projects
NVD CVE-1999-0656	Medium	1 Jan 1999	CWE-16	5.0	2
NVD CVE-2007-0998	Medium	20 Mar 2007	CWE-264	4.3	2
NVD CVE-2007-2768	Medium	21 May 2007	CWE-200	4.3	2
NVD CVE-2007-4998	Medium	31 Jan 2008	CWE-59	6.9	2
NVD CVE-2008-3844	High	27 Aug 2008	CWE-20	9.3	2

Figure 3: Dependency-Track “Grouped Vulnerabilities” tab of the “Vulnerability Audit” page



(a) Dashboard graphs with variants picker

	ID ↓	Severity ↓	SBOM Affected	Published Date ↓	Variants	Actions
☐	CVE-2026-35386	low	openssh@9.6p1	Unknown	yocto-spdx	Edit
☐	CVE-2026-35385	high	openssh@9.6p1	Unknown	yocto-spdx	Edit
☐	CVE-2026-35092	high	corosync@3.1.10	1 avr. 2026	default yocto-spdx	Edit
☐	CVE-2026-35091	high	corosync@3.1.10	Unknown	default yocto-spdx	Edit
☐	CVE-2026-34933	medium	avahi@0.8	3 avr. 2026	default	Edit
Total: 50						

1-50 / 1149 - Results per page: 50

First Previous 1 2 3 ... 23 Next Last

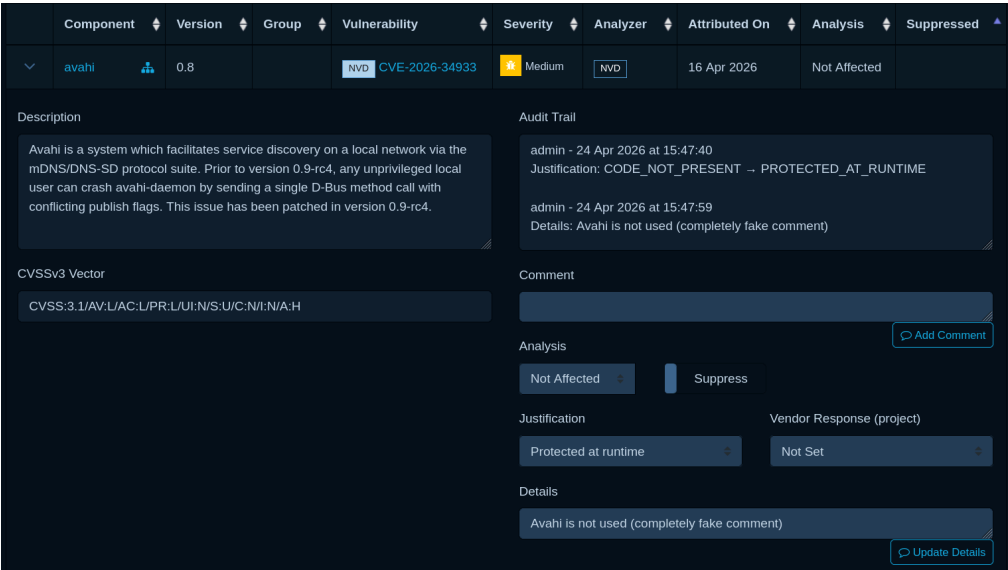
VulnScout v8.12

(b) Vulnerabilities tab with multiple variants

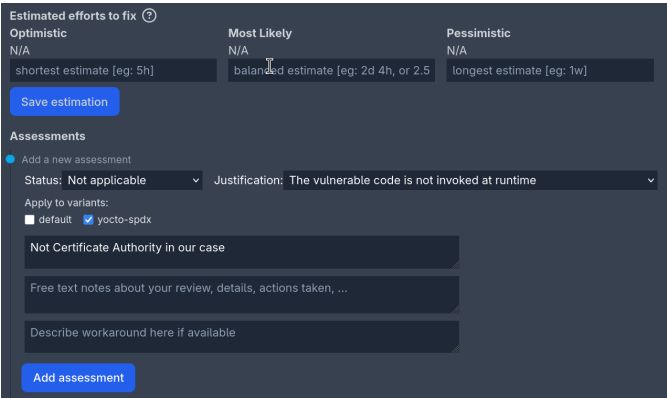
Figure 4: Variants in VulnScout

2.4. Assessments

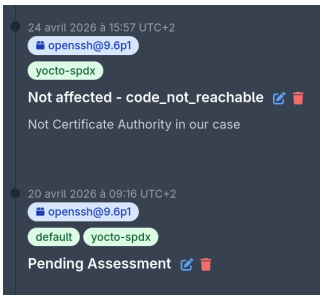
The workflow for assessing vulnerabilities are roughly the same in both tools: select a vulnerability, check its details, select a status (exploitable, false positive, etc.), add some details depending on the status you selected (why is not applicable?) then optionally add a comment on your assessment (see [Figure 5a](#) and [Figure 5b](#)). The assessment trail from VulnScout ([Figure 5c](#)) is more pleasant to work with on vulnerabilities that require lot of work: Dependency-Track’s audit trail is text-only and is a bit rough since it contains every single change ungrouped.



(a) Assessing a vulnerability in Dependency-Track



(b) Assessing a vulnerability in VulnScout



(c) Assessment trail in VulnScout

Figure 5: Vulnerability assessment screens in both tools

One can see in [Figure 5b](#) that VulnScout has a “time estimates” feature that can be quite interesting to gauge the time to spend in order to triage vulnerabilities. Those tracked times can then be exported in CSV thanks to the previously mentioned “Reports” feature ([Section 2.2](#)).

The variants feature mentioned in [Section 2.3](#) is also present in VulnScout’s assessment page: when assessing a vulnerability, the user can select to which variant the assessment applies using the check-

boxes (see Figure 5b). In Dependency-Track this is not possible, users have to completely duplicate the assessment in all versions.

Another similar advantage of VulnScout is the ability to bulk assess vulnerabilities: while in Dependency-Track you'd have to go on every single vulnerability and write the same thing over and over, in VulnScout you can simply select all the vulnerabilities you want and do your assessment from there (see Figure 6).

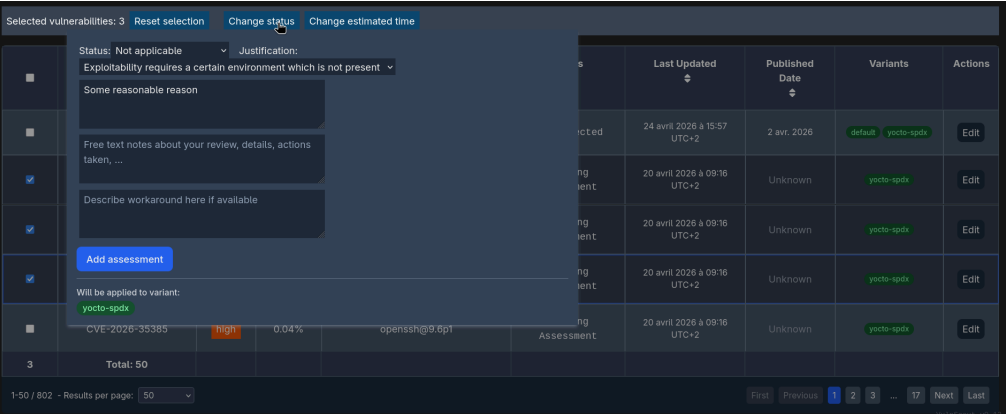


Figure 6: VulnScout bulk assessment dialog

2.5. Policies

Both tools have a way for users to setup a condition when vulnerabilities are “too important” and should ring a bell.

In VulnScout, the feature is meant to be used in a CI environment and is really simple: you run VulnScout with the `--match-condition` flag set to a condition and optionally a report that will be exported with the matched CVEs. The program will exit with a non-zero code if vulnerabilities matched the condition, making it extremely easy-to-use in CI pipelines. Concerning the conditions, they are logic expressions that can be as complex as necessary (see an example in [Listing 1](#)). For now, match conditions are purely a back-end feature and do not exist on the dashboard.

```
$ ./vulnscout \
--match-condition "((cvss >= 9.0 or (cvss >= 7.0 and epss >= 30%)) and (pending
== true or affected == true))" \
--report "match_condition.adoc"
# Vulnerability triggered fail condition: CVE-2016-0749
# Vulnerability triggered fail condition: CVE-2016-10642
# ...
# Report written: /scan/outputs/match_condition.adoc
```

Listing 1: A VulnScout execution with a match condition and a report

In contrast, in Dependency-Track the feature is fully integrated in the app: there is a “policy management” tab that allows users to define any number of policies with names, “violation states” (info, warn or fail) and conditions. The conditions cannot be as complex as in VulnScout: you only input a list of criterion that are AND-ed (see [Figure 7](#)). However, where this feature truly shines is how integrated it is with the rest of the app: there are graphs showing the amount of vulnerabilities violating the policies ([Figure 8a](#) and [Figure 8b](#)) and lists showing only them for easy assessment ([Figure 8c](#)). Dependency-Track policies are however not as easy to integrate in CI/CD pipelines since it would need communicating via the REST API to get a list of violations and handle the results ourselves, which is more complex than getting a failure exit code (more on than in [Section 2.7](#)).

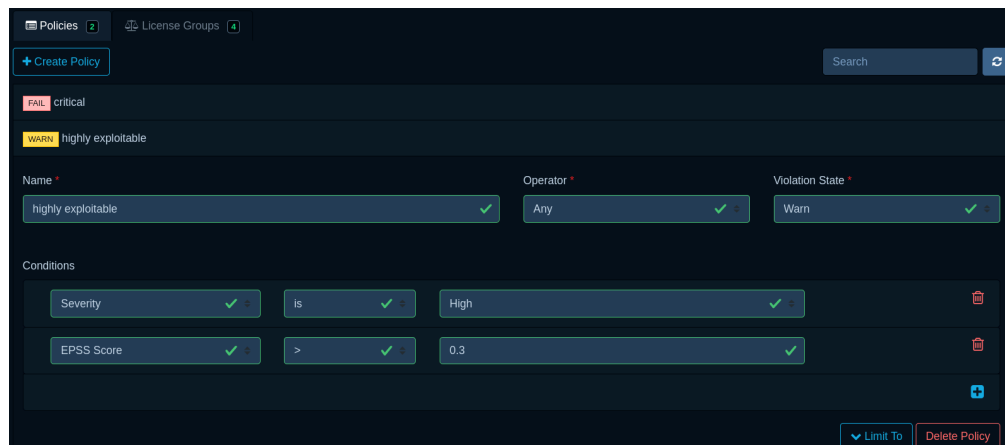
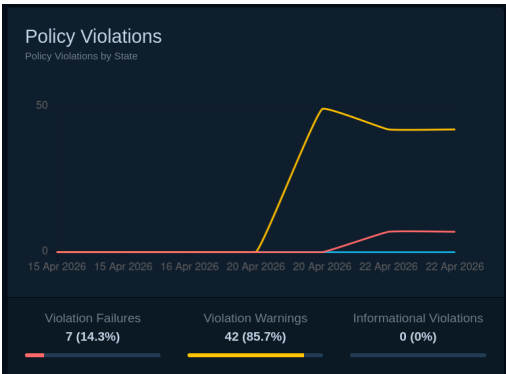


Figure 7: Dependency-Track “Policy Management” tab with two policies already set-up



(a) Policy violations evolution graph on the main dashboard

Project Name	Version	Latest	Classifier	Last BOM Import	BOM Format	Risk Score	Informational Violations: 0	Violation Warnings: 42	Violation Failures: 7	Policy Violations	Vulnerabilities
SEAPATH	1.2.0		Operating system	16 Apr 2026 at 15:56:59	CycloneDX 1.4	2055	Total: 49	42	7	180	251

(b) Policy violations summary in the project list

SEAPATH 1.2.0 LATEST VERSION

9180351120

View Details >

Overview

Components 401

Services 0

Dependency Graph 0

Audit Vulnerabilities 552 588

Exploit Predictions 552

Policy Violations 49 0 42 7

Show suppressed violations

Search

Refresh

Filter

	State	Risk Type	Policy Name	Component	Occurred On	Analysis	Suppressed
>	FAIL	Security	critical	wget 1.21.4	20 Apr 2026	-	
>	WARN	Security	highly exploitable	urllib3 2.2.2	20 Apr 2026	-	
>	WARN	Security	highly exploitable	unzip 6.0	20 Apr 2026	-	
>	WARN	Security	highly exploitable	unzip 6.0	20 Apr 2026	-	
>	WARN	Security	highly exploitable	syslog-ng 4.6.0	20 Apr 2026	-	

(c) Policy violations list for a specific project

Figure 8: Various Dependency-Track views where policy violations appear

2.6. Exploit Prediction

In order to predict if an exploit can happen and how important it can be, one has to look at both scores:

- CVSS⁶ (Common Vulnerability Scoring System): a score of how severe a vulnerability
- EPSS⁷ (Exploit Prediction Scoring System): a score of how likely a vulnerability can be exploited

Looking at the combination of both allows to gauge which vulnerabilities should be taken care of more importantly. Dependency-Track has a really useful visualization for this: a two-dimension chart that plots every vulnerability by their CVSS and EPSS (see Figure 9). The most important vulnerabilities can be directly seen at the top right. For now, VulnScout does not have such a visualization, but one can sort the vulnerabilities list by either EPSS or CVSS and find the most important ones at the top.

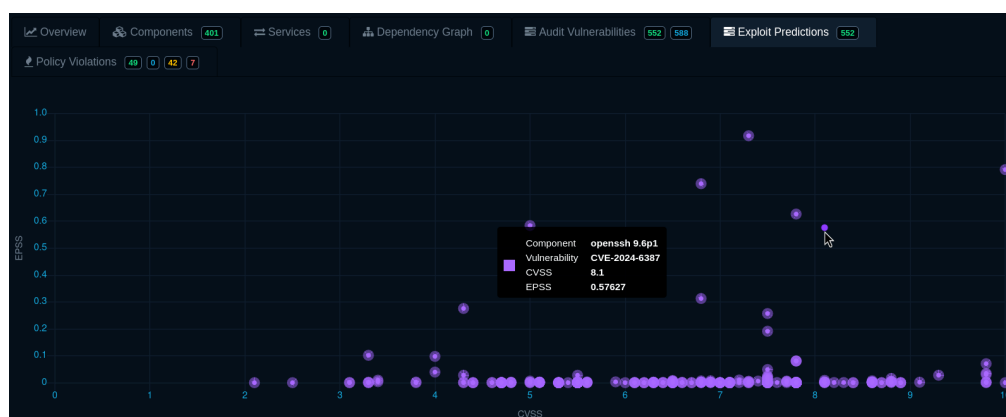


Figure 9: Dependency-Track “Exploit Predictions” tab

2.7. Integration with CIs

VulnScout is really easy to integrate in a CI thanks to its complete CLI.⁸ Since VulnScout is basically a container that can be spinned up in a few seconds, it can be soundly started directly in the CI environment without needing a particular cache. This makes it good to use in public CIs such as GitHub Actions [5]. You can find in Listing 2 a simple workflow that loads SPDX and Yocto cve-check files, generates reports and fails if some CVEs matched the fail condition.

```
./vulnscout --project demo --variant x86 \
  --add-spdx $(pwd)/example/spdx3/core-image-minimal-qemux86-64.rootfs.spdx.json \
  --add-cve-check $(pwd)/example/spdx3/core-image-minimal-qemux86-64.rootfs.json

./vulnscout \
  --match-condition "((cvss >= 9.0 or (cvss >= 7.0 and epss >= 30%)) and (pending
== true or affected == true)) \
  --report "summary.adoc" \
  --report "vulnerabilities.csv" \
  --report "match_condition.adoc"

# The previous command will fail if a CVE matched the condition, thus failing the
workflow.
```

Listing 2: Example of a simple CI workflow for VulnScout

⁶<https://www.first.org/cvss/>

⁷<https://www.first.org/epss/>

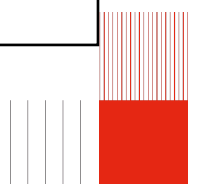
⁸<https://vulnscout.readthedocs.io/en/latest/vulnscout-script.html#non-interactive-mode-ci-automation>

Dependency-Track cannot be used in the same way since it requires its database mirror to be populated, which takes a long time. For it to be used in CI, the workflow must reference a deployed instance of Dependency-Track and use the REST API to interact with it, which means managing authentication via access tokens. Then, to do the actual interaction, there are several possibilities:

- use the official Jenkins plugin which does a really great job (uploads BOMs, displays vulnerabilities, reports policy violations...);
- use the official GitHub Action which is not actively maintained and only handles uploading BOMs;
- use third-party GitHub Actions which do more things... but are maintained by third parties, which is a security risk;
- making raw HTTP requests and handle the data received, which is more complex to write.

2.8. Integration with Yocto

As already stated in [Section 2.1](#), Dependency-Track cannot directly work on the output of a Yocto build, you need a separate layer to create CycloneDX files. On the contrary, VulnScout directly supports the files created by Yocto so they can be uploaded via the frontend or the CLI. But in order to ease even more integration with Yocto, Savoir-Faire Linux developed a custom `meta-vulnscout` layer[6] that includes all of the necessary scripts and classes to produce the best CVE check possible (for example by filtering kernel CVEs) and even a custom task to directly start VulnScout from inside Bitbake. This layer automatically sets up variants (see [Section 2.3](#)) based on the selected image/machine/distribution for easy integration.



3. Conclusion

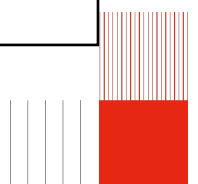
Both tools serve fundamentally different purposes. Dependency-Track is made to be deployed once in a centralized location (e.g. on a server in the company's internal network) and then accessed by everyone from there. It centralizes SBOMs, licenses, vulnerabilities and so on for every project of the company. It also has a really powerful "policies" feature that makes tracking important vulnerabilities easy.

On the other hand, VulnScout is more self-contained: users that want to see or assess vulnerabilities simply spin it up on their machine, load the SBOMs and previous assessment files and go on from there. It offers no mean of centralization, the goal is to be able to quickly get a working environment to assess vulnerabilities. This flexibility makes it easily usable in CI/CDs.

VulnScout truly shines for open-source and/or Yocto-based projects: its self-contained nature makes it easy for contributors and users to reproduce assessments, and its wide range of supported input formats contain the default output formats of native Yocto classes, making integration practically painless. Users wanting to use VulnScout directly in the Bitbake environment can even use the `meta-vulnscout` layer which eases out the setup to a breeze. Its tightly integrated variants feature makes assessing multiple different Yocto builds easier than Dependency-Track.

Bibliography

- [1] "VulnScout: The Easy-to-Use Tool for Securing Embedded Systems and Staying Compliant," Savoir-faire Linux. Accessed: Apr. 23, 2026. [Online]. Available: <https://blog.savoirfairelinux.com/en-ca/2025/vulnscout-the-easy-to-use-tool-for-securing-embedded-systems-and-staying-compliant/>
- [2] "Dependency-Track." Accessed: Apr. 22, 2026. [Online]. Available: <https://dependencytrack.org/>
- [3] The Yocto Project®, "Yocto Project Documentation." Accessed: Apr. 21, 2026. [Online]. Available: <https://docs.yoctoproject.org/>
- [4] *meta-cyclonedx*. Accessed: Apr. 22, 2026. [Online]. Available: <https://github.com/iris-GmbH/meta-cyclonedx>
- [5] C. Chandrasekara and P. Herath, *Hands-on GitHub Actions: Implement CI/CD with GitHub Action Workflows for Your Applications*. Berkeley, CA: Apress, 2021. doi: 10.1007/978-1-4842-6464-5.
- [6] *meta-vulnscout*. Accessed: Mar. 11, 2026. [Online]. Available: <https://github.com/savoirfairelinux/meta-vulnscout/>



Vulnerabilities Report

Legal Notice

This report is a summary of the vulnerabilities found in the project. Scan was made on unknown date and export of this report was made on 2026-05-29. Keep in mind that this report is not exhaustive and may not contain all vulnerabilities found in the project. Also, new vulnerabilities are discovered every day and thus the precision and accuracy of this report will decrease over time.

Summary

Number of vulnerabilities found: 669

Number of vulnerabilities present in this report: 669

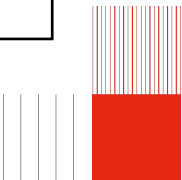
They are classified as follow:

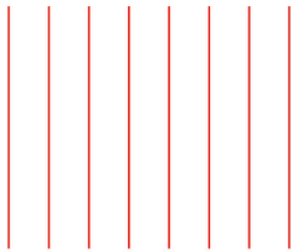
Status	Critical	High	Medium	Low	Unknown	Total
Pending investigation	5	39	53	3	1	101
Confirmed and exploitable	0	0	0	0	0	0
Fixed	0	0	0	0	182	182
Ignored or false positive	19	67	167	7	126	386

Most critical vulnerabilities

The following list use EPSS ranking to outline the vulnerabilities which are most used. The EPSS ranking is a metric that takes into account the number of known exploits, the ease of exploitation, the impact of the vulnerability, and the availability of a fix.

Here should be a list of critical vulnerabilities. Currently there are none.





Le projet porte sur la sécurisation de SEAPATH en mettant en place un suivi automatisé des vulnérabilités pour ses variantes Yocto et Debian. Le but est de fournir aux mainteneurs des rapports exploitables afin d'améliorer la sécurité d'un produit utilisé dans des infrastructures critiques.

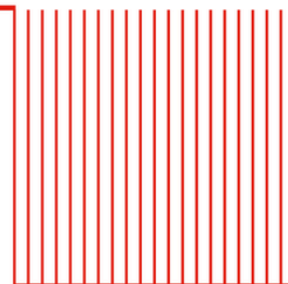
Plusieurs approches ont été étudiées et implémentées pour détecter les vulnérabilités, notamment à l'aide de SBOMs. Des analyses ont été menées pour améliorer la qualité des informations extraites et réduire les faux positifs. Le travail a donné lieu à des contributions dans de grands projets open source.

L'ensemble a été intégré dans des pipelines d'intégration continue, avec des contrôles périodiques et des rapports automatisés pour faciliter le suivi et la remontée d'alertes tout en étant transparent et reproductible.

The project focuses on securing SEAPATH by implementing automated vulnerability tracking for its Yocto and Debian variants. The goal is to provide maintainers with actionable reports to improve the security of a product used in critical infrastructure.

Several approaches were studied and implemented to detect vulnerabilities, notably using SBOMs. Analyses were carried out to improve the quality of extracted information and reduce false positives. The work resulted in contributions to major open-source projects.

All of this was integrated into continuous integration pipelines, with periodic checks and automated reports to facilitate monitoring and the escalation of alerts while remaining transparent and reproducible.



INSA Rennes

20 avenue des Buttes de Coësmes
CS 70839

35708 Rennes cedex 7

Tél : + 33 (0)2 23 23 82 00

www.insa-rennes.fr



INSA INSTITUT NATIONAL
DES SCIENCES
APPLIQUÉES
RENNES


MINISTÈRE
DE L'ENSEIGNEMENT
SUPÉRIEUR
ET DE LA RECHERCHE
*Liberté
Égalité
Fraternité*